# 10. Alberi binari

## Programmazione II

Ferruccio Damiani

a.a. 2023/2024

Università degli Studi di Torino - Dipartimento di Informatica

## Attenzione!

©2024 Copyright for this slides by Ferruccio Damiani. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

https://creativecommons.org/licenses/by/4.0/

## Si noti che

questi lucidi sono parzialmente basati sul libro di testo del corso "P. J. Deilel & H. M. Deitel, *Il linguaggio C - Fondamenti e tecniche di programmazione*, Pearson, 2022".

# Table of contents


1. Definizione astratta e definizione concreta di albero binario
2. Alcune definizioni e proprietà sugli Alberi Binari[1]
3. Alberi di Ricerca Binari (ARB) – Binary Search Trees (BSTs)
4. Principio di induzione strutturale sugli Alberi Binari
5. Manipolazione degli alberi binari collegati
6. Visita (degli elementi) di un albero binario
7. Operazioni sugli alberi di ricerca binari
8. Esercizi (in preparazione dell'esame)
9. Take-home message(s)


[1] I contenuti delle sides in questa sezione sono in parte basati sulle note della Dr. Maria Rita Di Berardini e della Prof. Emanuela Merelli (UniCam).

# Definizione astratta e definizione concreta di albero binario

# Definizione astratta di albero binario, notazioni e terminologia

## Definizione (albero binario – una definizione astratta)

Un albero binario bt (di elementi di tipo E) è:

- o l'albero vuoto, indicato con $/$;
- oppure una tripla $\langle$left,elem,right$\rangle$ dove:
  - elem è un valore (di tipo E), e
  - left, right sono alberi binari (di elementi di tipo E).

Ad esempio, la rappresentazione grafica dell'albero binario

$$\langle\langle/, 13, /\rangle, 27, \langle\langle/, 33, /\rangle, 42, \langle/, 48, /\rangle\rangle\rangle$$

è:

```
      27
    /    \
  13      42
        /    \
      33      48
```

# Definizione astratta di albero binario, notazioni e terminologia

**Definizione (albero binario – una definizione astratta)**

Un albero binario bt (di elementi di tipo E) è:

- o l'albero vuoto, indicato con $/$;
- oppure una tripla $\langle$left,elem,right$\rangle$ dove:
  - elem è un valore (di tipo E), e
  - left, right sono alberi binari (di elementi di tipo E).

Ad esempio, la rappresentazione grafica dell'albero binario

$$\langle\langle/, 13, /\rangle, 27, \langle\langle/, 33, /\rangle, 42, \langle/, 48, /\rangle\rangle\rangle$$

è:

```
      27
    /    \
  13      42
         /   \
       33     48
```

L'elemento elem è detto la radice (root) di bt, l'albero binario left è detto il sottoalbero sinistro (left subtree) di bt, l'albero binario right è detto il sottoalbero destro (right subtree) di bt;

# Definizione astratta di albero binario, notazioni e terminologia

**Definizione (albero binario – una definizione astratta)**

Un albero binario bt (di elementi di tipo E) è:

- o l'albero vuoto, indicato con $/$;
- oppure una tripla $\langle$left,elem,right$\rangle$ dove:
  - elem è un valore (di tipo E), e
  - left, right sono alberi binari (di elementi di tipo E).

Ad esempio, la rappresentazione grafica dell'albero binario

$$\langle\langle/, 13, /\rangle, 27, \langle\langle/, 33, /\rangle, 42, \langle/, 48, /\rangle\rangle\rangle$$

è:

```
      27
    /    \
  13      42
         /   \
       33     48
```

L'elemento elem è detto la radice (root) di bt, l'albero binario left è detto il sottoalbero sinistro (left subtree) di bt, l'albero binario right è detto il sottoalbero destro (right subtree) di bt;

- la radice di left è detta il figlio sinistro (left child) dell'elemento elem;
- la radice di right è detta il figlio destro (left child) dell'elemento elem;
- i figli di elem sono detti fratelli (siblings);
- un elemento senza figli è detto foglia (leaf);
- un elemento è interno (inner) se non è ne la radice nè una foglia.

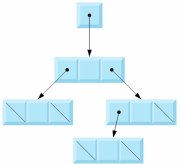
**L'albero binario è un esempio di struttura dati ricorsiva (o autoreferenziale)** – PERCHÉ?

# Una definizione concreta di albero binario: l'albero binario collegato, impl. in C

```
// Il tipo degli alberi binari di (elementi di tipo) T
typedef ... T; // Il tipo degli elementi

typedef struct treeNode *Tree; // Il tipo degli alberi binari
struct treeNode {
    Tree left;
    T data;
    Tree right;
};
```

```
// Scritto senza usare "typedef"
struct treeNode {
    struct treeNode *left;
    ... data;
    struct treeNode *right;
};
```

```
// I tipo degli alberi binari di (elementi di tipo) int
typedef struct node *IntTree;
struct intTreeNode {
    IntTree left;
    int data;
    IntTree right;
};
```

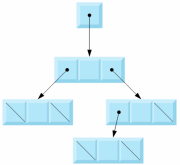


# Una definizione concreta di albero binario: l'albero binario collegato, impl. in C

```
// Il tipo degli alberi binari di (elementi di tipo) T
typedef ... T; // Il tipo degli elementi

typedef struct treeNode *Tree; // Il tipo degli alberi binari
struct treeNode {
    Tree left;
    T data;
    Tree right;
};
```

```
// Scritto senza usare "typedef"
struct treeNode {
    struct treeNode *left;
    ... data;
    struct treeNode *right;
};
```

```
// I tipo degli alberi binari di (elementi di tipo) int
typedef struct node *IntTree;
struct intTreeNode {
    IntTree left;
    int data;
    IntTree right;
};
```



**Definizione (albero binario – una definizione astratta)**

Un albero binario bt (di elementi di tipo T) è:

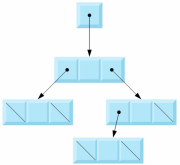
- o l'albero vuoto, indicato con /
- oppure una tripla ⟨left,elem,right⟩ dove:
  - elem è un valore (di tipo T), e
  - left, right sono alberi binari (di elementi di tipo T).

# Una definizione concreta di albero binario: l'albero binario collegato, impl. in C

```
// Il tipo degli alberi binari di (elementi di tipo) T
typedef ... T; // Il tipo degli elementi

typedef struct treeNode *Tree; // Il tipo degli alberi binari
struct treeNode {
    Tree left;
    T data;
    Tree right;
};
```

```
// Scritto senza usare "typedef"
struct treeNode {
    struct treeNode *left;
    ... data;
    struct treeNode *right;
};
```

```
// I tipo degli alberi binari di (elementi di tipo) int
typedef struct node *IntTree;
struct intTreeNode {
    IntTree left;
    int data;
    IntTree right;
};
```



**Definizione (albero binario – una definizione astratta)**

Un albero binario bt (di elementi di tipo T) è:

- o l'albero vuoto, indicato con /
- oppure una tripla ⟨left,elem,right⟩ dove:
  - elem è un valore (di tipo T), e
  - left, right sono alberi binari (di elementi di tipo T).

**Definizione (albero binario – una definizione concreta in C)**

Una variabile bt di tipo Tree rappresenta un albero binario (di elementi di tipo T) se contiene:

- o l'albero vuoto, rappresentato dal valore *NULL*
- oppure un puntatore ad un struct treeNode dove:
  - data contiene un valore (di tipo T), e
  - left, right rappresentano, rispettivamente, due alberi binari (di elementi di tipo T).

[a] I contenuti delle sides in questa sezione sono in parte basati sulle note della Dr. Maria Rita Di Berardini e della Prof. Emanuela Merelli (UniCam).

# Alcune definizioni e proprietà sugli Alberi Binari[a]

# Cammini in un albero, profondità di un nodo e altezza di un albero

## Definizione (cammini in un albero)

Un cammino da un nodo n ad un altro nodo m è una sequenza di nodi connessi da archi che portano da n ad m. Sia n un nodo di un albero binario T con radice r.

- Un qualsiasi nodo m lungo un cammino dalla radice r ad n è detto antenato di n.
- Se m è antenato di n, allora n è discendente di m (ogni nodo è antenato e discendente di se stesso); se $m \neq n$, allora m è antenato proprio di n, e n è discendente proprio di m.

# Cammini in un albero, profondità di un nodo e altezza di un albero

## Definizione (cammini in un albero)

Un cammino da un nodo n ad un altro nodo m è una sequenza di nodi connessi da archi che portano da n ad m. Sia n un nodo di un albero binario T con radice r.

- Un qualsiasi nodo m lungo un cammino dalla radice r ad n è detto antenato di n.
- Se m è antenato di n, allora n è discendente di m (ogni nodo è antenato e discendente di se stesso); se $m \neq n$, allora m è antenato proprio di n, e n è discendente proprio di m.

## Def. (profondità di un nodo e altezza di un albero)

- In un albero la profondità di un nodo è la lunghezza del cammino che porta dalla radice a quel nodo (cioè il numero di archi tra la radice ed il nodo – la radice ha quindi profondità 0).
- L'altezza dell'albero è la profondità del nodo di profondità massima (l'albero vuoto ha altezza -1).

# Cammini in un albero, profondità di un nodo e altezza di un albero

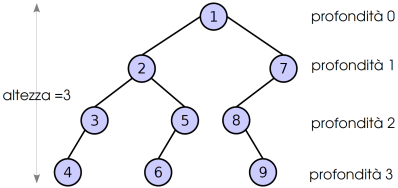
## Definizione (cammini in un albero)

Un cammino da un nodo n ad un altro nodo m è una sequenza di nodi connessi da archi che portano da n ad m. Sia n un nodo di un albero binario T con radice r.

- Un qualsiasi nodo m lungo un cammino dalla radice r ad n è detto antenato di n.
- Se m è antenato di n, allora n è discendente di m (ogni nodo è antenato e discendente di se stesso); se $m \neq n$, allora m è antenato proprio di n, e n è discendente proprio di m.

## Def. (profondità di un nodo e altezza di un albero)

- In un albero la profondità di un nodo è la lunghezza del cammino che porta dalla radice a quel nodo (cioè il numero di archi tra la radice ed il nodo – la radice ha quindi profondità 0).
- L'altezza dell'albero è la profondità del nodo di profondità massima (l'albero vuoto ha altezza -1).



# Grado di un nodo, alberi binari completi, quasi completi e pseudo quasi completi

**Definizione (grado di un nodo, alberi binari completi e (pseudo) quasi completi)**

Il grado di un nodo è il numero di figli di quel nodo. Un albero binario si dice completo se:

- tutte le foglie hanno la stessa profondità h; e
- tutti i nodi interni hanno grado 2 (cioè, hanno esattamente due figli).

Un albero binario è pseudo quasi completo se tutti i livelli, tranne al più l'ultimo, sono completi; è quasi completo se le eventuali foglie mancanti nell'ultimo livello sono consecutive a partire dall'ultima a destra.

# Grado di un nodo, alberi binari completi, quasi completi e pseudo quasi completi

**Definizione (grado di un nodo, alberi binari completi e (pseudo) quasi completi)**

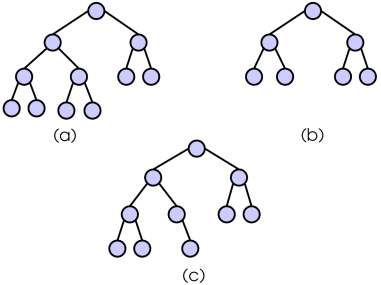
Il grado di un nodo è il numero di figli di quel nodo. Un albero binario si dice completo se:

- tutte le foglie hanno la stessa profondità h; e
- tutti i nodi interni hanno grado 2 (cioè, hanno esattamente due figli).

Un albero binario è pseudo quasi completo se tutti i livelli, tranne al più l'ultimo, sono completi; è quasi completo se le eventuali foglie mancanti nell'ultimo livello sono consecutive a partire dall'ultima a destra.

Ad esempio:

- L'albero (a) non è completo, ma è quasi completo.
- L'albero (b) è completo.
- L'albero (c) non è completo, non è quasi completo, ma è pseudo quasi completo.



# Alberi binari degeneri e completamente sbilanciati

**Definizione (alberi binari degeneri e completamente sbilanciati)**

Un albero binario (con n nodi e altezza h) si dice

- degenere se ogni nodo ha alpiù un figlio (da cui segue h == n-1);
- completamente sbilanciato
  - a sinistra se ogni nodo ha alpiù il figlio sinistro;
  - a destra se ogni nodo ha alpiù il figlio destro.

# Alberi binari degeneri e completamente sbilanciati

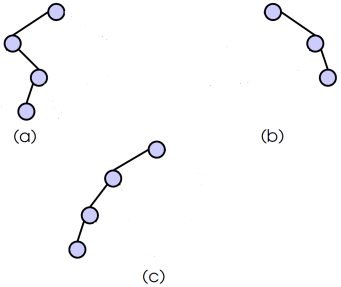
**Definizione (alberi binari degeneri e completamente sbilanciati)**

Un albero binario (con n nodi e altezza h) si dice

- degenere se ogni nodo ha alpiù un figlio (da cui segue h == n-1);
- completamente sbilanciato
  - a sinistra se ogni nodo ha alpiù il figlio sinistro;
  - a destra se ogni nodo ha alpiù il figlio destro.

Ad esempio:

- L'albero (a) è degenere.
- L'albero (b) è (degenere e) completamente sbilanciato a destra.
- L'albero (c) è (degenere e) completamente sbilanciato a sinistra.



# Proprietà degli alberi binari completi

## Proposizione 1 (numero di nodi di un albero binario completo)

Un albero binario completo di altezza $h$ ha $2^{h+1} - 1$ nodi.

**Dimostrazione.** La prova è per induzione su $h$.

- **Caso base** ($h = -1$)**.** Un albero binario completo di altezza $h = -1$ ha zero nodi (è vuoto). Siccome $2^{-1+1} - 1 = 0$, il caso è verificato.
- **Caso base** ($h = 0$)**.** Un albero binario completo di altezza $h = 0$ ha un solo nodo (la radice). Siccome $2^{0+1} - 1 = 1$, il caso è verificato.
- **Passo induttivo:** ($h > 1$)**.** Assumiamo (per ipotesi induttiva) che un albero binario completo di altezza $h$ abbia $2^{h+1} - 1$ nodi, e dimostriamo che un albero binario T completo e di altezza $h + 1$ ha $2^{(h+1)+1} - 1 = 2^{h+2} - 1$ nodi. L'albero T ha (per definizione) la radice, più un sottoalbero sinistro $T_s$ e un sottoalbero destro $T_d$ entrambi completi e di altezza h. Per ipotesi induttiva $\#\text{nodi}(T_d) = \#\text{nodi}(T_s) = 2^{h+1} - 1$. Quindi: $\#\text{nodi}(\text{T}) = 1 + \#\text{nodi}(T_s) + \#\text{nodi}(T_d) =$
$1 + (2^{h+1} - 1) + (2^{h+1} - 1) = (2 \cdot 2^{h+1}) - 1 = 2^{h+2} - 1$.

# Proprietà degli alberi binari completi

## Proposizione 1 (numero di nodi di un albero binario completo)

Un albero binario completo di altezza $h$ ha $2^{h+1} - 1$ nodi.

**Dimostrazione.** La prova è per induzione su $h$.

- **Caso base** ($h = -1$)**.** Un albero binario completo di altezza $h = -1$ ha zero nodi (è vuoto). Siccome $2^{-1+1} - 1 = 0$, il caso è verificato.
- **Caso base** ($h = 0$)**.** Un albero binario completo di altezza $h = 0$ ha un solo nodo (la radice). Siccome $2^{0+1} - 1 = 1$, il caso è verificato.
- **Passo induttivo:** ($h > 1$)**.** Assumiamo (per ipotesi induttiva) che un albero binario completo di altezza $h$ abbia $2^{h+1} - 1$ nodi, e dimostriamo che un albero binario T completo e di altezza $h + 1$ ha $2^{(h+1)+1} - 1 = 2^{h+2} - 1$ nodi. L'albero T ha (per definizione) la radice, più un sottoalbero sinistro $T_s$ e un sottoalbero destro $T_d$ entrambi completi e di altezza h. Per ipotesi induttiva $\#\text{nodi}(T_d) = \#\text{nodi}(T_s) = 2^{h+1} - 1$. Quindi: $\#\text{nodi}(\text{T}) = 1 + \#\text{nodi}(T_s) + \#\text{nodi}(T_d) = 1 + (2^{h+1} - 1) + (2^{h+1} - 1) = (2 \cdot 2^{h+1}) - 1 = 2^{h+2} - 1$.

## Esercizio 2 (numero di foglie di un albero binario completo)

Dimostrate (di nuovo per induzione su $h$) che un albero binario completo di altezza $h$ ha esattamente $2^h$ foglie.

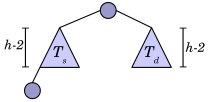
# Proprietà degli alberi binari quasi completi

## Proposizione 3 (numero di nodi di un albero binario quasi completo)

Sia T un albero binario quasi completo di altezza $h$. Allora: $2^h \leq \#\text{nodi}(\text{T}) \leq 2^{h+1} - 1$.

**Dimostrazione.** Per dimostrare questo risultato abbiamo bisogno di determinare il numero massimo e minimo di nodi di un albero binario quasi completo di altezza $h$.

- Il numero massimo di nodi che un albero binario quasi completo di altezza $h$ può avere è pari al numero di nodi di albero binario completo di altezza $h$, ossia $\max = 2^{h+1} - 1$ (vedi Proposizione 1).
- L'albero binario quasi completo di altezza $h$ con numero minimo di nodi la seguente forma (dove $T_s$ e $T_d$ sono degli alberi binari completi di altezza $h-2$):




Allora: $\#\text{nodi}(T) = 1 + 1 + \#\text{nodi}(T_s) + \#\text{nodi}(T_d)$ dove $\#\text{nodi}(T_s) = \#\text{nodi}(T_d) = 2^{h-1} - 1$ (questo perché sia $T_s$ che $T_d$ sono degli alberi binari completi di altezza $h-2$). Quindi, $\#\text{nodi}(T) = 2 + (2^{h-1} - 1) + (2^{h-1} - 1) = 2 \cdot 2^{h-1} = 2^h$.

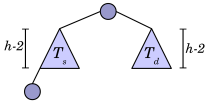
# Proprietà degli alberi binari quasi completi

## Proposizione 3 (numero di nodi di un albero binario quasi completo)

Sia T un albero binario quasi completo di altezza $h$. Allora: $2^h \leq \#\text{nodi}(T) \leq 2^{h+1} - 1$.

**Dimostrazione.** Per dimostrare questo risultato abbiamo bisogno di determinare il numero massimo e minimo di nodi di un albero binario quasi completo di altezza $h$.

- Il numero massimo di nodi che un albero binario quasi completo di altezza $h$ può avere è pari al numero di nodi di albero binario completo di altezza $h$, ossia $\max = 2^{h+1} - 1$ (vedi Proposizione 1).
- L'albero binario quasi completo di altezza $h$ con numero minimo di nodi la seguente forma (dove $T_s$ e $T_d$ sono degli alberi binari completi di altezza $h-2$):




Allora: $\#\text{nodi}(T) = 1 + 1 + \#\text{nodi}(T_s) + \#\text{nodi}(T_d)$ dove $\#\text{nodi}(T_s) = \#\text{nodi}(T_d) = 2^{h-1} - 1$ (questo perché sia $T_s$ che $T_d$ sono degli alberi binari completi di altezza $h-2$). Quindi, $\#\text{nodi}(T) = 2 + (2^{h-1} - 1) + (2^{h-1} - 1) = 2 \cdot 2^{h-1} = 2^h$.

## Proposizione 4 (altezza di un albero binario quasi completo)

L'altezza di un albero binario quasi completo $T$ con $n$ nodi è $h = \lfloor \log n \rfloor$.

**Dimostrazione.** Per la Proposizione 3 abbiamo che $2^h \leq n \leq 2^{h+1} - 1 < 2^{h+1}$, quindi $h \leq \log n < h + 1$, cioè $h \leq \lfloor \log n \rfloor < h + 1$, da cui segue $h = \lfloor \log n \rfloor$.

# Alberi di Ricerca Binari (ARB) – Binary Search Trees (BSTs)

# Alberi di Ricerca Binari (ARB) – Binary Search Trees (BSTs)

**Definizione (albero di ricerca binario (ARB))**

Un albero di ricerca binario è un albero binario bt tale che: (i) gli elementi appartengono ad un insieme totalmente ordinato rispetto ad relazione $\leq$ (minore o uguale); e (ii) per ogni elemento k di bt:

- tutti gli elementi del sottoalbero sinistro di k sono $\leq$ k; e
- tutti gli elementi del sottoalbero destro di k sono $\geq$ k.

Di norma consideremo ARB in cui tutti gli elementi sono distinti – nel qual caso, nella definizione precedente, si possono sostituire $\leq$ e $\geq$ con $<$ e $>$ (str. min. e str. mag.).

# Alberi di Ricerca Binari (ARB) – Binary Search Trees (BSTs)

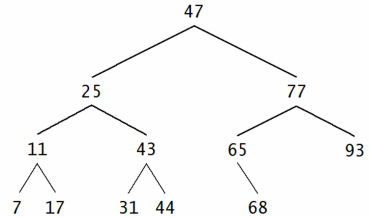
**Definizione (albero di ricerca binario (ARB))**

Un albero di ricerca binario è un albero binario bt tale che: (i) gli elementi appartengono ad un insieme totalmente ordinato rispetto ad relazione $\leq$ (minore o uguale); e (ii) per ogni elemento k di bt:

- tutti gli elementi del sottoalbero sinistro di k sono $\leq$ k; e
- tutti gli elementi del sottoalbero destro di k sono $\geq$ k.

Di norma consideremo ARB in cui tutti gli elementi sono distinti – nel qual caso, nella definizione precedente, si possono sostituire $\leq$ e $\geq$ con $<$ e $>$ (str. min. e str. mag.).

Un esempio di ARB di numeri naturali:



# Principio di induzione strutturale sugli Alberi Binari

# Principio di induzione strutturale sugli Alberi Binari

## Principio di induzione strutturale sugli Alberi Binari

Se, data una proprietà P(bt) sugli alberi binari[2]

- Riusciamo a dimostrare che tale proprietà vale per l'abero vuoto / **(caso base)**.
- Assumendo che essa valga per gli alberi left e right allora, dato un qualunque elemento elem, riusciamo a dimostrate che essa vale anche per l'albero binario ⟨left,elem,right⟩ **(caso induttivo)**.

allora abbiamo dimostrato che P vale per qualunque albero binario.

[2] Ad es.: $P_{size}$(bt) = "la funzione C size – con intestazione `size_t size(Tree bt)` – restituisce il numero di nodi dell'albero binario passatogli come argomento".
3

# Principio di induzione strutturale sugli Alberi Binari

## Principio di induzione strutturale sugli Alberi Binari

Se, data una proprietà P(bt) sugli alberi binari[2]

- Riusciamo a dimostrare che tale proprietà vale per l'abero vuoto / **(caso base)**.
- Assumendo che essa valga per gli alberi left e right allora, dato un qualunque elemento elem, riusciamo a dimostrate che essa vale anche per l'albero binario ⟨left,elem,right⟩ **(caso induttivo)**.

allora abbiamo dimostrato che P vale per qualunque albero binario.

Il **Principio di Induzione su Strutture Dati Ricorsive** (di cui Principio di Induzione Strutturale su Alberi Binari è un caso particolare) è una generlizzazione del principio di Induzione sui Numeri Naturali. Ricopre, nello sviluppo di algoritmi ricorsivi che operano su liste, un ruolo analogo a quello ricoperto dalla **Metodologia di sviluppo basata sull'Invariante di ciclo** nello sviluppo di algoritmi iterativi.[3]

[2] Ad es.: $P_{size}$(bt) = "la funzione C size – con intestazione `size_t size(Tree bt)` – restituisce il numero di nodi dell'albero binario passatogli come argomento".

[3] Per dimostrare proprietà di alberi binari può essere talvolta conveniente usare, invece del principio di induzione strutturale, il principio di induzione sui numeri naturali (ad es. sull'altezza dell'albero).

# Manipolazione degli alberi binari collegati

# Manipolazione degli alberi binari collegati

Nelle prossime slides illustreremo gli algorimi di base per manipolare alberi binari collegati e ARB collegati, usando alberi di interi:

```
typedef struct intTreeNode IntTreeNode, *IntTree;
struct intTreeNode {
    IntTree left;
    int data;
    IntTree right;
};
```

alberi di caratteri:

```
typedef struct charNode CharTreeNode, *CharTree;
struct charTreeNode {
    CharTree left;
    char data;
    CharTree right;
};
```

e (per quelle operazioni che non dipendono dal tipo degli elementi) alberi di elementi di un tipo `T` non specificato:

```
typedef ... T;
typedef struct treeNode TreeNode, *Tree;
struct treeNode {
    Tree left;
    T data;
    Tree right;
};
```

# Visita (degli elementi) di un albero binario

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);`
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);`
- Visita post-order – `void postOrder(Tree tree);`

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);`
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);`
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);`
- Visita post-order – `void postOrder(Tree tree);`

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);`
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

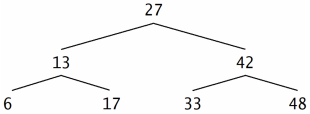



**Fig. 1** Un albero (di ricerca) binario.

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);` – visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro (in particolare, visita gli elementi di un ARB in ordine).
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);`
- Visita post-order – `void postOrder(Tree tree);`

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);`
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

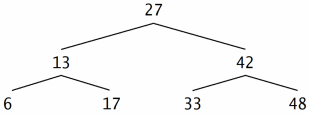



**Fig. 1** Un albero (di ricerca) binario.

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);` – visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro (in particolare, visita gli elementi di un ARB in ordine). Ad es., per l'ARB in Fig. 1: 6 13 17 27 33 42 48.
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);`
- Visita post-order – `void postOrder(Tree tree);`

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);`
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

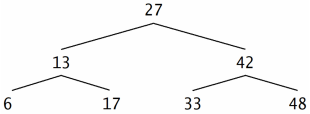



**Fig. 1** Un albero (di ricerca) binario.

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);` – visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro (in particolare, visita gli elementi di un ARB in ordine). Ad es., per l'ARB in Fig. 1: 6 13 17 27 33 42 48.
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro.
- Visita post-order – `void postOrder(Tree tree);`

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);`
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

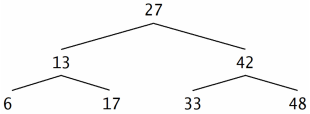



**Fig. 1** Un albero (di ricerca) binario.

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);` – visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro (in particolare, visita gli elementi di un ARB in ordine). Ad es., per l'ARB in Fig. 1: 6 13 17 27 33 42 48.
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro. Ad es., per l'ARB in Fig. 1: 27 13 6 17 42 33 48.
- Visita post-order – `void postOrder(Tree tree);`

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);`
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

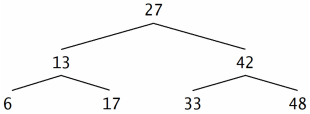



**Fig. 1** Un albero (di ricerca) binario.

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);` – visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro (in particolare, visita gli elementi di un ARB in ordine). Ad es., per l'ARB in Fig. 1: 6 13 17 27 33 42 48.
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro. Ad es., per l'ARB in Fig. 1: 27 13 6 17 42 33 48.
- Visita post-order – `void postOrder(Tree tree);` – visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice.

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);`
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

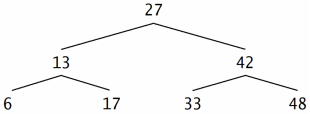



**Fig. 1** Un albero (di ricerca) binario.

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);` – visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro (in particolare, visita gli elementi di un ARB in ordine). Ad es., per l'ARB in Fig. 1: 6 13 17 27 33 42 48.
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro. Ad es., per l'ARB in Fig. 1: 27 13 6 17 42 33 48.
- Visita post-order – `void postOrder(Tree tree);` – visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice. Ad es., per l'ARB in Fig. 1: 6 17 13 33 48 42 27 .

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);`
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

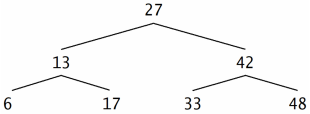



**Fig. 1** Un albero (di ricerca) binario.

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);` – visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro (in particolare, visita gli elementi di un ARB in ordine). Ad es., per l'ARB in Fig. 1: 6 13 17 27 33 42 48.
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro. Ad es., per l'ARB in Fig. 1: 27 13 6 17 42 33 48.
- Visita post-order – `void postOrder(Tree tree);` – visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice. Ad es., per l'ARB in Fig. 1: 6 17 13 33 48 42 27 .

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);` – visita l'albero per livelli, iniziando dalla radice, visitando i nodi di ciascun livello da sinistra verso destra (si appoggia ad una coda).
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

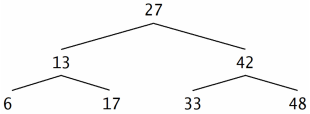



**Fig. 1** Un albero (di ricerca) binario.

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);` – visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro (in particolare, visita gli elementi di un ARB in ordine). Ad es., per l'ARB in Fig. 1: 6 13 17 27 33 42 48.
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro. Ad es., per l'ARB in Fig. 1: 27 13 6 17 42 33 48.
- Visita post-order – `void postOrder(Tree tree);` – visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice. Ad es., per l'ARB in Fig. 1: 6 17 13 33 48 42 27 .

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);` – visita l'albero per livelli, iniziando dalla radice, visitando i nodi di ciascun livello da sinistra verso destra (si appoggia ad una coda). Ad es., per l'ARB in Fig. 1: 27 13 42 6 17 33 48.
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);`

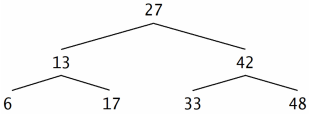



**Fig. 1** Un albero (di ricerca) binario.

# Visita (degli elementi) di un albero binario

Vedremo 3 algoritmi di visita ricorsivi:

- Visita in-order – `void inOrder(Tree tree);` – visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro (in particolare, visita gli elementi di un ARB in ordine). Ad es., per l'ARB in Fig. 1: 6 13 17 27 33 42 48.
- Visita pre-order o in profondità (depth-first) – `void preOrder(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro. Ad es., per l'ARB in Fig. 1: 27 13 6 17 42 33 48.
- Visita post-order – `void postOrder(Tree tree);` – visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice. Ad es., per l'ARB in Fig. 1: 6 17 13 33 48 42 27 .

E 2 algoritmi di visita iterativi:

- Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);` – visita l'albero per livelli, iniziando dalla radice, visitando i nodi di ciascun livello da sinistra verso destra (si appoggia ad una coda). Ad es., per l'ARB in Fig. 1: 27 13 42 6 17 33 48.
- Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);` – (si appoggia ad una pila).

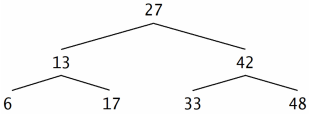



**Fig. 1** Un albero (di ricerca) binario.

# Visita in-order

Visita in-order: visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro.

```
/** @brief Assume: tree e' un albero. Esegue: visita in-order dei nodi di tree.
*/
void inOrder(Tree tree) {
    if (tree == NULL) return;

    inOrder(tree->left);
    /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
    inOrder(tree->right);
}
```

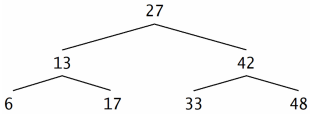
4

# Visita in-order

Visita in-order: visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro.

```
/** @brief Assume: tree e' un albero. Esegue: visita in-order dei nodi di tree.
*/
void inOrder(Tree tree) {
    if (tree == NULL) return;

    inOrder(tree->left);
    /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
    inOrder(tree->right);
}
```

Ad es., per l'ARB:




stampa: 6 13 17 27 33 42 48.

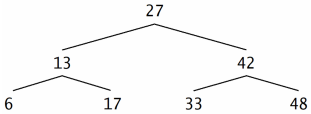
4

# Visita in-order

Visita in-order: visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro.

```
/** @brief Assume: tree e' un albero. Esegue: visita in-order dei nodi di tree.
*/
void inOrder(Tree tree) {
    if (tree == NULL) return;

    inOrder(tree->left);
    /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
    inOrder(tree->right);
}
```

Ad es., per l'ARB:




stampa: 6 13 17 27 33 42 48.

Esempi concreti di operazioni di visita in-order sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):[4]

- void printPreOrder(CharTree tree); – stampa gli elementi dell'albero in-order (OK, per questa la soluzione ve l'ho di fatto già data).
- size_t size(Tree tree); – restituisce il numero degli elementi dell'albero.
- size_t height(Tree tree); – restituisce l'altezza dell'albero.
- int sumIntTree(IntTree tree); – restituisce la somma degli elementi dell'albero.
- size_t count(Tree tree, _Bool (*predicate)(T e)); – restituisce il numero degli elementi dell'albero che soddisfano il predicato 'predicate'.

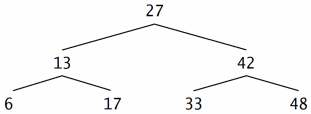
[4] Magari qualcuna la vediamo in Laboratorio.

# Visita in-order

Visita in-order: visita il sottoalbero sinistro, visita la radice, visita il sottoalbero destro.

```
/** @brief Assume: tree e' un albero. Esegue: visita in-order dei nodi di tree.
*/
void inOrder(Tree tree) {
    if (tree == NULL) return;

    inOrder(tree->left);
    /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
    inOrder(tree->right);
}
```

Ad es., per l'ARB:




stampa: 6 13 17 27 33 42 48.

Esempi concreti di operazioni di visita in-order sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):[4]

- void printPreOrder(CharTree tree); – stampa gli elementi dell'albero in-order (OK, per questa la soluzione ve l'ho di fatto già data).
- size_t size(Tree tree); – restituisce il numero degli elementi dell'albero.
- size_t height(Tree tree); – restituisce l'altezza dell'albero.
- int sumIntTree(IntTree tree); – restituisce la somma degli elementi dell'albero.
- size_t count(Tree tree, _Bool (*predicate)(T e)); – restituisce il numero degli elementi dell'albero che soddisfano il predicato 'predicate'.

Complessità computazionale delle operazioni di visita precedenti, rispetto al numero di elementi n e all'altezza h dell'albero

- tempo: $\Theta(n)$,
- spazio: $\Theta(h)$, in tutti i casi (pessimo, ottimo). – tenete presente che nel caso pessimo (albero degenere) h == n-1; mentre nel caso ottimo (albero pseudo quasi completo) h == $\log_2 n$.

[4] Magari qualcuna la vediamo in Laboratorio.

# Visita pre-order o in profondità (depth-first)

Visita pre-order: visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro.

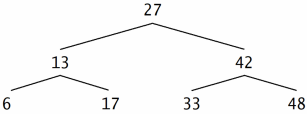
```
/** @brief Assume: tree e' un albero. Esegue: visita pre-order dei nodi di tree.
*/
void preOrder(Tree tree) {
    if (tree == NULL) return;

    /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
    preOrder(tree->left);
    preOrder(tree->right);
}
```

# Visita pre-order o in profondità (depth-first)

Visita pre-order: visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro.

```
/** @brief Assume: tree e' un albero. Esegue: visita pre-order dei nodi di tree.
*/
void preOrder(Tree tree) {
    if (tree == NULL) return;

    /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
    preOrder(tree->left);
    preOrder(tree->right);
}
```

Ad es., per l'ARB:



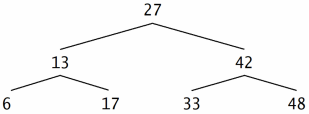

stampa: 27 13 6 17 42 33 48.

# Visita pre-order o in profondità (depth-first)

Visita pre-order: visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro.

```
/** @brief Assume: tree e' un albero. Esegue: visita pre-order dei nodi di tree.
*/
void preOrder(Tree tree) {
    if (tree == NULL) return;

    /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
    preOrder(tree->left);
    preOrder(tree->right);
}
```

Ad es., per l'ARB:




stampa: 27 13 6 17 42 33 48.

Esempi concreti di operazioni di visita pre-order sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):

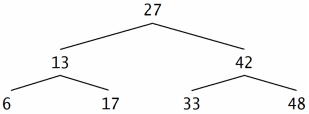
- void printPreOrder(CharTree tree); – stampa gli elementi dell'albero in pre-order (OK, per questa la soluzione ve l'ho di fatto già data).
- Gli stessi esempi elencati per la visita in-order.

# Visita pre-order o in profondità (depth-first)

Visita pre-order: visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro.

```
/** @brief Assume: tree e' un albero. Esegue: visita pre-order dei nodi di tree.
*/
void preOrder(Tree tree) {
    if (tree == NULL) return;

    /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
    preOrder(tree->left);
    preOrder(tree->right);
}
```

Ad es., per l'ARB:




stampa: 27 13 6 17 42 33 48.

Esempi concreti di operazioni di visita pre-order sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):

- void printPreOrder(CharTree tree); – stampa gli elementi dell'albero in pre-order (OK, per questa la soluzione ve l'ho di fatto già data).
- Gli stessi esempi elencati per la visita in-order.

Complessità computazionale delle operazioni di visita precedenti, rispetto al numero di elementi n e all'altezza h dell'albero – tempo: $\Theta(n)$, spazio: $\Theta(h)$, in tutti i casi (pessimo, ottimo).

# Visita post-order

Visita post-order: visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice.

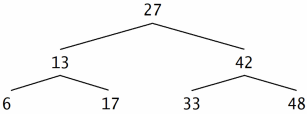
```
/** @brief Assume: tree e' un albero. Esegue: visita post-order dei nodi di tree.
*/
void postOrder(Tree tree) {
    if (tree == NULL) return;

    postOrder(tree->left);
    postOrder(tree->right);
     /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
}
```

# Visita post-order

Visita post-order: visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice.

```
/** @brief Assume: tree e' un albero. Esegue: visita post-order dei nodi di tree.
*/
void postOrder(Tree tree) {
    if (tree == NULL) return;

    postOrder(tree->left);
    postOrder(tree->right);
     /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
}
```

Ad es., per l'ARB:



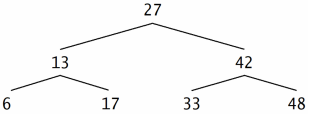

stampa: 6 17 13 33 48 42 27.

# Visita post-order

Visita post-order: visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice.

```
/** @brief Assume: tree e' un albero. Esegue: visita post-order dei nodi di tree.
*/
void postOrder(Tree tree) {
    if (tree == NULL) return;

    postOrder(tree->left);
    postOrder(tree->right);
     /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
}
```

Ad es., per l'ARB:




stampa: 6 17 13 33 48 42 27.

Esempi concreti di operazioni di visita post-order sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):

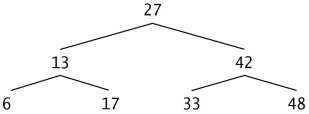
- void printPostOrder(CharTree tree); – stampa gli elementi dell'albero in post-order (OK, per questa la soluzione ve l'ho di fatto già data).
- Gli stessi esempi elencati per la visita in-order.

# Visita post-order

Visita post-order: visita il sottoalbero sinistro, visita il sottoalbero destro, visita la radice.

```
/** @brief Assume: tree e' un albero. Esegue: visita post-order dei nodi di tree.
*/
void postOrder(Tree tree) {
    if (tree == NULL) return;

    postOrder(tree->left);
    postOrder(tree->right);
     /* Inserire qui il codice che visita la radice di tree; ad es.: printf("%3d", tree->data); */
}
```

Ad es., per l'ARB:




stampa: 6 17 13 33 48 42 27.

Esempi concreti di operazioni di visita post-order sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):

- void printPostOrder(CharTree tree); – stampa gli elementi dell'albero in post-order (OK, per questa la soluzione ve l'ho di fatto già data).
- Gli stessi esempi elencati per la visita in-order.

Complessità computazionale delle operazioni di visita precedenti, rispetto al numero di elementi n e all'altezza h dell'albero – tempo: $\Theta(n)$, spazio: $\Theta(h)$, in tutti i casi (pessimo, ottimo).

# Visita pre-order o in profondità (depth-first) – VERSIONE ITERATIVA

Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro (si appoggia ad una pila).

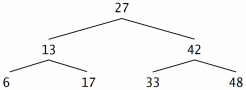
```
/** @brief Assume: tree e' un albero. Esegue: visita pre-order dei nodi di tree.
*/
void depthFirst(Tree tree) {
    if (tree == NULL) return;

    VoidPtrStackADT stack = mkVoidPtrStackADT();
    push(stack, tree);

    // I.C.: Se !isEmpty(stack), allora: (1) i nodi di tree gia' visitati sono quelli che precedono il nodo top(stack) nell'ordine pre-order e sono stati visitati nell'ordine pre-order; e
   //**** (2) stack contiene, sotto a top(stack), in ordine decresente di profondita', i figli destri degli antenati di top(stack) che non sono antenati di top(stack).
   //**** Se isEmpty(stack), allora tutti i nodi di tree sono stati visitati nell'ordine pre-order.
    while (isEmpty(stack)) {
        Tree nodePtr = pop(stack);
        /* Inserire qui il codice che visita in nodo *nodePtr; ad es.: printf("%3d", nodePtr->data); */
        if (nodePtr->right != NULL) { push(stack, nodePtr->right); }
        if (nodePtr->left != NULL) { push(stack, nodePtr->left); /* Sara' il possimo nodo a essere visitato. */ }
    }
}
```

# Visita pre-order o in profondità (depth-first) – VERSIONE ITERATIVA

Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro (si appoggia ad una pila).

```
/** @brief Assume: tree e' un albero. Esegue: visita pre-order dei nodi di tree.
*/
void depthFirst(Tree tree) {
    if (tree == NULL) return;

    VoidPtrStackADT stack = mkVoidPtrStackADT();
    push(stack, tree);

    // I.C.: Se !isEmpty(stack), allora: (1) i nodi di tree gia' visitati sono quelli che precedono il nodo top(stack) nell'ordine pre-order e sono stati visitati nell'ordine pre-order; e
   //**** (2) stack contiene, sotto a top(stack), in ordine decresente di profondita', i figli destri degli antenati di top(stack) che non sono antenati di top(stack).
   //**** Se isEmpty(stack), allora tutti i nodi di tree sono stati visitati nell'ordine pre-order.
    while (isEmpty(stack)) {
        Tree nodePtr = pop(stack);
        /* Inserire qui il codice che visita in nodo *nodePtr; ad es.: printf("%3d", nodePtr->data); */
        if (nodePtr->right != NULL) { push(stack, nodePtr->right); }
        if (nodePtr->left != NULL) { push(stack, nodePtr->left); /* Sara' il possimo nodo a essere visitato. */ }
    }
}
```

Ad es., per l'ARB:



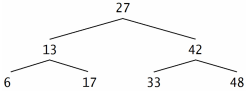

stampa: 27 13 6 17 42 33 48.

# Visita pre-order o in profondità (depth-first) – VERSIONE ITERATIVA

Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro (si appoggia ad una pila).

```
/** @brief Assume: tree e' un albero. Esegue: visita pre-order dei nodi di tree.
*/
void depthFirst(Tree tree) {
    if (tree == NULL) return;

    VoidPtrStackADT stack = mkVoidPtrStackADT();
    push(stack, tree);

    // I.C.: Se !isEmpty(stack), allora: (1) i nodi di tree gia' visitati sono quelli che precedono il nodo top(stack) nell'ordine pre-order e sono stati visitati nell'ordine pre-order; e
   //**** (2) stack contiene, sotto a top(stack), in ordine decresente di profondita', i figli destri degli antenati di top(stack) che non sono antenati di top(stack).
   //**** Se isEmpty(stack), allora tutti i nodi di tree sono stati visitati nell'ordine pre-order.
    while (isEmpty(stack)) {
        Tree nodePtr = pop(stack);
        /* Inserire qui il codice che visita in nodo *nodePtr; ad es.: printf("%3d", nodePtr->data); */
        if (nodePtr->right != NULL) { push(stack, nodePtr->right); }
        if (nodePtr->left != NULL) { push(stack, nodePtr->left); /* Sara' il possimo nodo a essere visitato. */ }
    }
}
```

Ad es., per l'ARB:




stampa: 27 13 6 17 42 33 48.

Esempi concreti di operazioni di visita pre-order sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):

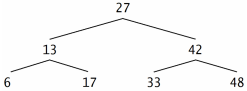
- void printDepthFirst(CharTree tree); – stampa gli elementi dell'albero in pre-order (OK, per questa la soluzione ve l'ho di fatto già data).
- Gli stessi esempi elencati per la visita in-order.

# Visita pre-order o in profondità (depth-first) – VERSIONE ITERATIVA

Versione iterativa della visita pre-order o in profondità (depth-first) – `void dephFirst(Tree tree);` – visita la radice, visita il sottoalbero sinistro, visita il sottoalbero destro (si appoggia ad una pila).

```
/** @brief Assume: tree e' un albero. Esegue: visita pre-order dei nodi di tree.
*/
void depthFirst(Tree tree) {
    if (tree == NULL) return;

    VoidPtrStackADT stack = mkVoidPtrStackADT();
    push(stack, tree);

    // I.C.: Se !isEmpty(stack), allora: (1) i nodi di tree gia' visitati sono quelli che precedono il nodo top(stack) nell'ordine pre-order e sono stati visitati nell'ordine pre-order; e
   //**** (2) stack contiene, sotto a top(stack), in ordine decresente di profondita', i figli destri degli antenati di top(stack) che non sono antenati di top(stack).
   //**** Se isEmpty(stack), allora tutti i nodi di tree sono stati visitati nell'ordine pre-order.
    while (isEmpty(stack)) {
        Tree nodePtr = pop(stack);
        /* Inserire qui il codice che visita in nodo *nodePtr; ad es.: printf("%3d", nodePtr->data); */
        if (nodePtr->right != NULL) { push(stack, nodePtr->right); }
        if (nodePtr->left != NULL) { push(stack, nodePtr->left); /* Sara' il possimo nodo a essere visitato. */ }
    }
}
```

Ad es., per l'ARB:




stampa: 27 13 6 17 42 33 48.

Esempi concreti di operazioni di visita pre-order sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):

- void printDepthFirst(CharTree tree); – stampa gli elementi dell'albero in pre-order (OK, per questa la soluzione ve l'ho di fatto già data).
- Gli stessi esempi elencati per la visita in-order.

Complessità computazionale delle operazioni di visita precedenti, rispetto al numero di elementi n e all'altezza h dell'albero

- tempo: $\Theta(n)$,
- spazio: $\Theta(h)$, in tutti i casi (pessimo, ottimo).

# Visita in ampiezza (breadth-first)

Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);` – visita l'albero nell'ordine "Livello-Sinistra-Destra (LSD)"; ovvero: per livelli, dal livello 0, visitando i nodi di ciascun livello da sinistra verso destra (si appoggia ad una coda).

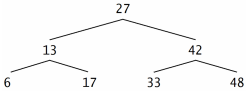
```
/** @brief Assume: tree e' un albero. Esegue: visita in ampiezza dei nodi di tree.
*/
void depthFirst(Tree tree) {
    if (tree == NULL) return;

    VoidPtrQueueADT queue = mkVoidPtrQueueADT();
    enqueue(queue, tree);

   // I.C.: Se !isEmpty(queue), allora: (1) i nodi di tree gia' visitati sono quelli che precedono il nodo front(queue) nell'ordine LSD e sono stati visitati nell'ordine LSD; e
   //**** (2) livello(front(queue)) <= livello(rear(queue)) <= livello(front(queue)) + 1.
   //**** Se !isEmpty(queue) e rear(queue) != tree, allora queue contiene nell'ordine LSD i nodi di tree che seguono nell'ordine LSD l'ultimo nodo visitato,
   //**** fino al [nodo rear(queue) che e' il] figlio piu' a destra dell'ultimo dei nodi visitati che ha almeno un figlio.
   //**** Se isEmpty(queue), allora tutti i nodi di tree sono stati visitati nell'ordine LSD.
    while (!isEmpty(queue)) {
        Tree nodePtr = dequeue(queue);
        /* Inserire qui il codice che visita in nodo *nodePtr; ad es.: printf("%3d", nodePtr->data); */
        if (nodePtr->left != NULL) { enqueue(queue, nodePtr->left); }
        if (nodePtr->right != NULL) { enqueue(queue, nodePtr->right); }
    }
}
```

# Visita in ampiezza (breadth-first)

Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);` – visita l'albero nell'ordine "Livello-Sinistra-Destra (LSD)"; ovvero: per livelli, dal livello 0, visitando i nodi di ciascun livello da sinistra verso destra (si appoggia ad una coda).

```
/** @brief Assume: tree e' un albero. Esegue: visita in ampiezza dei nodi di tree.
*/
void depthFirst(Tree tree) {
    if (tree == NULL) return;

    VoidPtrQueueADT queue = mkVoidPtrQueueADT();
    enqueue(queue, tree);

   // I.C.: Se !isEmpty(queue), allora: (1) i nodi di tree gia' visitati sono quelli che precedono il nodo front(queue) nell'ordine LSD e sono stati visitati nell'ordine LSD; e
   //**** (2) livello(front(queue)) <= livello(rear(queue)) <= livello(front(queue)) + 1.
   //**** Se !isEmpty(queue) e rear(queue) != tree, allora queue contiene nell'ordine LSD i nodi di tree che seguono nell'ordine LSD l'ultimo nodo visitato,
   //**** fino al [nodo rear(queue) che e' il] figlio piu' a destra dell'ultimo dei nodi visitati che ha almeno un figlio.
   //**** Se isEmpty(queue), allora tutti i nodi di tree sono stati visitati nell'ordine LSD.
    while (!isEmpty(queue)) {
        Tree nodePtr = dequeue(queue);
        /* Inserire qui il codice che visita in nodo *nodePtr; ad es.: printf("%3d", nodePtr->data); */
        if (nodePtr->left != NULL) { enqueue(queue, nodePtr->left); }
        if (nodePtr->right != NULL) { enqueue(queue, nodePtr->right); }
    }
}
```

Ad es., per l'ARB:



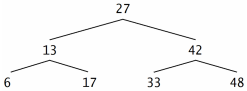

stampa: 27 13 42 6 17 33 48.

# Visita in ampiezza (breadth-first)

Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);` – visita l'albero nell'ordine "Livello-Sinistra-Destra (LSD)"; ovvero: per livelli, dal livello 0, visitando i nodi di ciascun livello da sinistra verso destra (si appoggia ad una coda).

```
/** @brief Assume: tree e' un albero. Esegue: visita in ampiezza dei nodi di tree.
*/
void depthFirst(Tree tree) {
    if (tree == NULL) return;

    VoidPtrQueueADT queue = mkVoidPtrQueueADT();
    enqueue(queue, tree);

   // I.C.: Se !isEmpty(queue), allora: (1) i nodi di tree gia' visitati sono quelli che precedono il nodo front(queue) nell'ordine LSD e sono stati visitati nell'ordine LSD; e
   //**** (2) livello(front(queue)) <= livello(rear(queue)) <= livello(front(queue)) + 1.
   //**** Se !isEmpty(queue) e rear(queue) != tree, allora queue contiene nell'ordine LSD i nodi di tree che seguono nell'ordine LSD l'ultimo nodo visitato,
   //**** fino al [nodo rear(queue) che e' il] figlio piu' a destra dell'ultimo dei nodi visitati che ha almeno un figlio.
   //**** Se isEmpty(queue), allora tutti i nodi di tree sono stati visitati nell'ordine LSD.
    while (!isEmpty(queue)) {
        Tree nodePtr = dequeue(queue);
        /* Inserire qui il codice che visita in nodo *nodePtr; ad es.: printf("%3d", nodePtr->data); */
        if (nodePtr->left != NULL) { enqueue(queue, nodePtr->left); }
        if (nodePtr->right != NULL) { enqueue(queue, nodePtr->right); }
    }
}
```

Ad es., per l'ARB:




stampa: 27 13 42 6 17 33 48.

Esempi concreti di operazioni di visita in ampiezza sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):

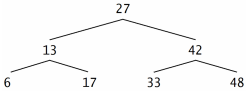
- void printDepthFirst(CharTree tree); – stampa gli elementi dell'albero in ampiezza (OK, per questa la soluzione ve l'ho di fatto già data).
- Gli stessi esempi elencati per la visita in-order.

# Visita in ampiezza (breadth-first)

Visita in ampiezza (breadth-first) – `void breadthFirst(Tree tree);` – visita l'albero nell'ordine "Livello-Sinistra-Destra (LSD)"; ovvero: per livelli, dal livello 0, visitando i nodi di ciascun livello da sinistra verso destra (si appoggia ad una coda).

```
/** @brief Assume: tree e' un albero. Esegue: visita in ampiezza dei nodi di tree.
*/
void depthFirst(Tree tree) {
    if (tree == NULL) return;

    VoidPtrQueueADT queue = mkVoidPtrQueueADT();
    enqueue(queue, tree);

   // I.C.: Se !isEmpty(queue), allora: (1) i nodi di tree gia' visitati sono quelli che precedono il nodo front(queue) nell'ordine LSD e sono stati visitati nell'ordine LSD; e
   //**** (2) livello(front(queue)) <= livello(rear(queue)) <= livello(front(queue)) + 1.
   //**** Se !isEmpty(queue) e rear(queue) != tree, allora queue contiene nell'ordine LSD i nodi di tree che seguono nell'ordine LSD l'ultimo nodo visitato,
   //**** fino al [nodo rear(queue) che e' il] figlio piu' a destra dell'ultimo dei nodi visitati che ha almeno un figlio.
   //**** Se isEmpty(queue), allora tutti i nodi di tree sono stati visitati nell'ordine LSD.
    while (!isEmpty(queue)) {
        Tree nodePtr = dequeue(queue);
        /* Inserire qui il codice che visita in nodo *nodePtr; ad es.: printf("%3d", nodePtr->data); */
        if (nodePtr->left != NULL) { enqueue(queue, nodePtr->left); }
        if (nodePtr->right != NULL) { enqueue(queue, nodePtr->right); }
    }
}
```

Ad es., per l'ARB:




stampa: 27 13 42 6 17 33 48.

Esempi concreti di operazioni di visita in ampiezza sono (LE LASCIO SCRIVERE A VOI, COME ESERCIZIO):

- void printDepthFirst(CharTree tree); – stampa gli elementi dell'albero in ampiezza (OK, per questa la soluzione ve l'ho di fatto già data).
- Gli stessi esempi elencati per la visita in-order.

Complessità computazionale delle operazioni di visita precedenti, rispetto al numero di elementi n dell'albero – tempo: $\Theta(n)$, spazio: $\Theta(n)$, in tutti i casi (pessimo, ottimo).

# Operazioni sugli alberi di ricerca binari

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);`
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);`
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);`
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);`

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);`
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);`
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);`

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL). Ad es., per l'ARB in Fig. 1, restituisce: 11.
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);`
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);`
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);`

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

**Fig. 1** Un ARB.

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL). Ad es., per l'ARB in Fig. 1, restituisce: 11.
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);`
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);`
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);`

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

**Fig. 1** Un ARB.

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL). Ad es., per l'ARB in Fig. 1, restituisce: 11.
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);`
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);`

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

**Fig. 1** Un ARB.

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL). Ad es., per l'ARB in Fig. 1, restituisce: 11.
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY. Ad es., per l'ARB in Fig. 1, l'inserimento di 44 produce l'ARB in Fig. 2.
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);`
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);`

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

**Fig. 1** Un ARB.

**Fig. 2** Un altro ARB.

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL). Ad es., per l'ARB in Fig. 1, restituisce: 11.
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY. Ad es., per l'ARB in Fig. 1, l'inserimento di 44 produce l'ARB in Fig. 2.
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);`

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

**Fig. 1** Un ARB.

**Fig. 2** Un altro ARB.

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL). Ad es., per l'ARB in Fig. 1, restituisce: 11.
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY. Ad es., per l'ARB in Fig. 1, l'inserimento di 44 produce l'ARB in Fig. 2.
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL). Ad es., per l'ARB in Fig. 2, restituisce 11 e produce l'ARB in Fig. 3.
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);`

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

**Fig. 1** Un ARB.

**Fig. 2** Un altro ARB.

**Fig. 3** Ancora un altro ARB.

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL). Ad es., per l'ARB in Fig. 1, restituisce: 11.
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY. Ad es., per l'ARB in Fig. 1, l'inserimento di 44 produce l'ARB in Fig. 2.
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL). Ad es., per l'ARB in Fig. 2, restituisce 11 e produce l'ARB in Fig. 3.
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

**Fig. 1** Un ARB.

**Fig. 2** Un altro ARB.

**Fig. 3** Ancora un altro ARB.

# Operazioni sugli alberi di ricerca binari

Vedremo 5 algoritmi iterativi:

- Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL). Ad es., per l'ARB in Fig. 1, restituisce: 11.
- Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.
- Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY. Ad es., per l'ARB in Fig. 1, l'inserimento di 44 produce l'ARB in Fig. 2.
- Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL). Ad es., per l'ARB in Fig. 2, restituisce 11 e produce l'ARB in Fig. 3.
- Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0. Ad es., per l'ARB in Fig. 2: (i) l'eliminazione di 44 produce l'ARB in Fig. 1; mentre l'eliminazione di 25 produce l'ARB in Fig. 4.

E 2 algoritmi ricorsivi:

- Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);`
- Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);`

**Fig. 1** Un ARB.

**Fig. 2** Un altro ARB.

**Fig. 3** Ancora un altro ARB.

**Fig. 4** E un altro ancora.

# RICERCA DEL MINIMO – versione ITERATIVA

Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: tree e' un ARB. Restituisce: il minimo elemento di un ARB non vuoto tree (termina il programma con exit(-1) se tree == NULL)
*/
T findMinBSTiter(Tree tree) {
    if (tree == NULL) { exit(-1); }

    Tree nodePtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: Il minimo elemento di tree e' il minimo elemento di nodePtr
   //**** (e se nodePtr->left == NULL, allora nodePtr->data e' il minimo elemento di tree).
    while ( /* Condizione */ ) { /* Istruzioni che producono un avvicinamento alla soluzione */ }

    return /* Valore dedotto da (I.C. && !Condizione) */ ;
}
```

# Ricerca del minimo – versione ITERATIVA

Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: tree e' un ARB. Restituisce: il minimo elemento di un ARB non vuoto tree (termina il programma con exit(-1) se tree == NULL)
*/
T findMinBSTiter(Tree tree) {
    if (tree == NULL) { exit(-1); }

    Tree nodePtr = /* Valore che garantisce una "partenza corretta" */;

   // I.C.: Il minimo elemento di tree e' il minimo elemento di nodePtr
   //**** (e se nodePtr->left == NULL, allora nodePtr->data e' il minimo elemento di tree).
    while ( /* Condizione */ ) { nodePtr = nodePtr->left; }

    return /* Valore dedotto da (I.C. && !Condizione) */ ;
}
```

# Ricerca del minimo – versione ITERATIVA

Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: tree e' un ARB. Restituisce: il minimo elemento di un ARB non vuoto tree (termina il programma con exit(-1) se tree == NULL)
*/
T findMinBSTiter(Tree tree) {
    if (tree == NULL) { exit(-1); }

    Tree nodePtr = /* Valore che garantisce una "partenza corretta" */;

   // I.C.: Il minimo elemento di tree e' il minimo elemento di nodePtr
   //**** (e se nodePtr->left == NULL, allora nodePtr->data e' il minimo elemento di tree).
    while (nodePtr->left != NULL) { nodePtr = nodePtr->left; }

    return /* Valore dedotto da (I.C. && !Condizione) */ ;
}
```

# Ricerca del minimo – versione ITERATIVA

Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: tree e' un ARB. Restituisce: il minimo elemento di un ARB non vuoto tree (termina il programma con exit(-1) se tree == NULL)
*/
T findMinBSTiter(Tree tree) {
    if (tree == NULL) { exit(-1); }

    Tree nodePtr = /* Valore che garantisce una ''partenza corretta'' */ ;

   // I.C.: Il minimo elemento di tree e' il minimo elemento di nodePtr
   //**** (e se nodePtr->left == NULL, allora nodePtr->data e' il minimo elemento di tree).
    while (nodePtr->left != NULL) { nodePtr = nodePtr->left; }

    return nodePtr->data;
}
```

# Ricerca del minimo – versione ITERATIVA

Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: tree e' un ARB. Restituisce: il minimo elemento di un ARB non vuoto tree (termina il programma con exit(-1) se tree == NULL)
*/
T findMinBSTiter(Tree tree) {
    if (tree == NULL) { exit(-1); }

    Tree nodePtr = tree;

   // I.C.: Il minimo elemento di tree e' il minimo elemento di nodePtr
   //**** (e se nodePtr->left == NULL, allora nodePtr->data e' il minimo elemento di tree).
    while (nodePtr->left != NULL) { nodePtr = nodePtr->left; }

    return nodePtr->data;
}
```

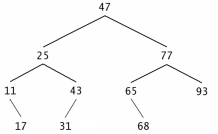
# Ricerca del minimo – versione ITERATIVA

Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: tree e' un ARB. Restituisce: il minimo elemento di un ARB non vuoto tree (termina il programma con exit(-1) se tree == NULL)
*/
T findMinBSTiter(Tree tree) {
    if (tree == NULL) { exit(-1); }

    Tree nodePtr = tree;

   // I.C.: Il minimo elemento di tree e' il minimo elemento di nodePtr
   //**** (e se nodePtr->left == NULL, allora nodePtr->data e' il minimo elemento di tree).
    while (nodePtr->left != NULL) { nodePtr = nodePtr->left; }

    return nodePtr->data;
}
```

Ad es., per l'ARB:




restituisce: 11.

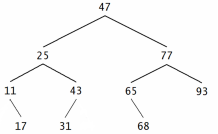
# Ricerca del minimo – versione ITERATIVA

Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: tree e' un ARB. Restituisce: il minimo elemento di un ARB non vuoto tree (termina il programma con exit(-1) se tree == NULL)
*/
T findMinBSTiter(Tree tree) {
    if (tree == NULL) { exit(-1); }

    Tree nodePtr = tree;

   // I.C.: Il minimo elemento di tree e' il minimo elemento di nodePtr
   //**** (e se nodePtr->left == NULL, allora nodePtr->data e' il minimo elemento di tree).
    while (nodePtr->left != NULL) { nodePtr = nodePtr->left; }

    return nodePtr->data;
}
```

Ad es., per l'ARB:




restituisce: 11.

Complessità computazionale rispetto all'altezza h dell'albero.

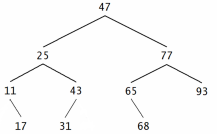
# Ricerca del minimo – versione ITERATIVA

Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: tree e' un ARB. Restituisce: il minimo elemento di un ARB non vuoto tree (termina il programma con exit(-1) se tree == NULL)
*/
T findMinBSTiter(Tree tree) {
    if (tree == NULL) { exit(-1); }

    Tree nodePtr = tree;

   // I.C.: Il minimo elemento di tree e' il minimo elemento di nodePtr
   //**** (e se nodePtr->left == NULL, allora nodePtr->data e' il minimo elemento di tree).
    while (nodePtr->left != NULL) { nodePtr = nodePtr->left; }

    return nodePtr->data;
}
```

Ad es., per l'ARB:




restituisce: 11.

Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (il minimo è nella radice);
- $\Theta(h)$, nel caso pessimo (il minimo è in una foglia di profondità massima).

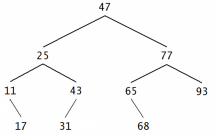
# Ricerca del minimo – versione ITERATIVA

Ricerca del minimo – `T findMinBSTiter(Tree tree);` – restituisce il minimo elemento di un ARB non vuoto tree (termina il programma se tree == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: tree e' un ARB. Restituisce: il minimo elemento di un ARB non vuoto tree (termina il programma con exit(-1) se tree == NULL)
*/
T findMinBSTiter(Tree tree) {
    if (tree == NULL) { exit(-1); }

    Tree nodePtr = tree;

   // I.C.: Il minimo elemento di tree e' il minimo elemento di nodePtr
   //**** (e se nodePtr->left == NULL, allora nodePtr->data e' il minimo elemento di tree).
    while (nodePtr->left != NULL) { nodePtr = nodePtr->left; }

    return nodePtr->data;
}
```

Ad es., per l'ARB:




restituisce: 11.

Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (il minimo è nella radice);
- $\Theta(h)$, nel caso pessimo (il minimo è in una foglia di profondità massima).

Spazio: $\Theta(1)$, in tutti casi (ottimo, pessimo).

# RICERCA DI UN ELEMENTO – versione ITERATIVA

Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: value e' presente in tree se—e—solo—se e' presente in nodePtr.
    while ( /* Condizione */ ) {

        /* Istruzioni che producono un avvicinamento alla soluzione */

    }
    return /* Valore dedotto da (I.C. && !Condizione) */ ;
}
```

# Ricerca di un elemento – versione ITERATIVA

Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: value e' presente in tree se—e—solo—se e' presente in nodePtr.
    while ( /* Condizione */ ) {
        if (value < nodePtr—>data) { nodePtr = nodePtr—>left; }
        else if (value > nodePtr—>data) { nodePtr = nodePtr—>right; }
        else return 1;
    }
    return /* Valore dedotto da (I.C. && !Condizione) */ ;
}
```

# Ricerca di un elemento – versione ITERATIVA

Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: value e' presente in tree se—e—solo—se e' presente in nodePtr.
    while (nodePtr != NULL) {
        if (value < nodePtr—>data) { nodePtr = nodePtr—>left; }
        else if (value > nodePtr—>data) { nodePtr = nodePtr—>right; }
        else return 1;
    }
    return /* Valore dedotto da (I.C. && !Condizione) */ ;
}
```

# Ricerca di un elemento – versione ITERATIVA

Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: value e' presente in tree se—e—solo—se e' presente in nodePtr.
    while (nodePtr != NULL) {
        if (value < nodePtr—>data) { nodePtr = nodePtr—>left; }
        else if (value > nodePtr—>data) { nodePtr = nodePtr—>right; }
        else return 1;
    }
    return 0;
}
```

# Ricerca di un elemento – versione ITERATIVA

Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = tree;

   // I.C.: value e' presente in tree se—e—solo—se e' presente in nodePtr.
    while (nodePtr != NULL) {
        if (value < nodePtr—>data) { nodePtr = nodePtr—>left; }
        else if (value > nodePtr—>data) { nodePtr = nodePtr—>right; }
        else return 1;
    }
    return 0;
}
```

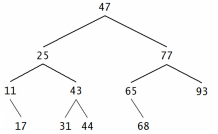
# Ricerca di un elemento – versione ITERATIVA

Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = tree;

   // I.C.: value e' presente in tree se—e—solo—se e' presente in nodePtr.
    while (nodePtr != NULL) {
        if (value < nodePtr—>data) { nodePtr = nodePtr—>left; }
        else if (value > nodePtr—>data) { nodePtr = nodePtr—>right; }
        else return 1;
    }
    return 0;
}
```

Ad es., per l'ARB:




la ricerca di 42 restituisce 0, mentre la ricerca di 43 restituisce 1.

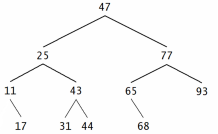
# Ricerca di un elemento – versione ITERATIVA

Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = tree;

  // I.C.: value e' presente in tree se—e—solo—se e' presente in nodePtr.
    while (nodePtr != NULL) {
        if (value < nodePtr—>data) { nodePtr = nodePtr—>left; }
        else if (value > nodePtr—>data) { nodePtr = nodePtr—>right; }
        else return 1;
    }
    return 0;
}
```

Ad es., per l'ARB:




la ricerca di 42 restituisce 0, mentre la ricerca di 43 restituisce 1.

Complessità computazionale rispetto all'altezza h dell'albero.

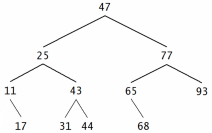
# Ricerca di un elemento – versione ITERATIVA

Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = tree;

  // I.C.: value e' presente in tree se—e—solo—se e' presente in nodePtr.
    while (nodePtr != NULL) {
        if (value < nodePtr—>data) { nodePtr = nodePtr—>left; }
        else if (value > nodePtr—>data) { nodePtr = nodePtr—>right; }
        else return 1;
    }
    return 0;
}
```

Ad es., per l'ARB:




la ricerca di 42 restituisce 0, mentre la ricerca di 43 restituisce 1.

Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (value è nella radice);
- $O(h)$, nel caso pessimo (value non è presente o è nella foglia di profondità massima).

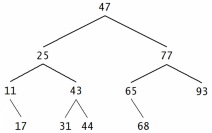
# Ricerca di un elemento – versione ITERATIVA

Ricerca di un elemento – `_Bool searchBSTiter(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBSTiter(IntTree tree, int value) {
    Tree nodePtr = tree;

  // I.C.: value e' presente in tree se—e—solo—se e' presente in nodePtr.
    while (nodePtr != NULL) {
        if (value < nodePtr—>data) { nodePtr = nodePtr—>left; }
        else if (value > nodePtr—>data) { nodePtr = nodePtr—>right; }
        else return 1;
    }
    return 0;
}
```

Ad es., per l'ARB:




la ricerca di 42 restituisce 0, mentre la ricerca di 43 restituisce 1.

Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (value è nella radice);
- $O(h)$, nel caso pessimo (value non è presente o è nella foglia di profondità massima).

Spazio: $\Theta(1)$, in tutti casi (ottimo, pessimo).

# INSERIMENTO DI UN ELEMENTO – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem.

```
/** @brief Assume: tree e' un ARB. Esegue: se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while ( /* Condizione */ ) {
            /* Istruzioni che producono un avvicinamento alla soluzione
               (ovvero nodePtrPtr punta alla cella in cui mettere l'indirizzo del nuovo nodo)
            */
    }


    /* Istruzioni che, assumendo (I.C. && !Condizione), inseriscono il nuovo elemento in una foglia */


    return INSERTED;
}
```

# Inserimento di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem.

```
/** @brief Assume: tree e' un ARB. Esegue: se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while ( /* Condizione */ ) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else { return ALREADYPRESENT; }
    }


    /* Istruzioni che, assumendo (I.C. && !Condizione), inseriscono il nuovo elemento in una foglia */


    return INSERTED;
}
```

# Inserimento di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem.

```
/** @brief Assume: tree e' un ARB. Esegue: se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else { return ALREADYPRESENT; }
    }


    /* Istruzioni che, assumendo (I.C. && !Condizione), inseriscono il nuovo elemento in una foglia */


    return INSERTED;
}
```

# Inserimento di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem.

```
/** @brief Assume: tree e' un ARB. Esegue: se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else { return ALREADYPRESENT; }
    }


    /* Istruzioni che, assumendo (I.C. && !Condizione), inseriscono il nuovo elemento in una foglia */


    return INSERTED;
}
```

# Inserimento di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem.

```
/** @brief Assume: tree e' un ARB. Esegue: se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)—>data) { nodePtrPtr = &((*nodePtrPtr)—>left); }
        else if (value > (*nodePtrPtr)—>data) { nodePtrPtr = &((*nodePtrPtr)—>right); }
        else { return ALREADYPRESENT; }
    }
    *nodePtrPtr = malloc(sizeof(IntTreeNode));
    if (*nodePtrPtr == NULL) { return OUTOFMEMORY; }
    (*nodePtrPtr)—>data = value;
    (*nodePtrPtr)—>left = NULL;
    (*nodePtrPtr)—>right = NULL;
    return INSERTED;
}
```

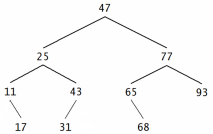
# Inserimento di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

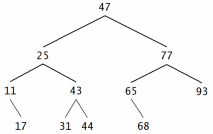
Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem.

```
/** @brief Assume: tree e' un ARB. Esegue: se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)—>data) { nodePtrPtr = &((*nodePtrPtr)—>left); }
        else if (value > (*nodePtrPtr)—>data) { nodePtrPtr = &((*nodePtrPtr)—>right); }
        else { return ALREADYPRESENT; }
    }
    *nodePtrPtr = malloc(sizeof(IntTreeNode));
    if (*nodePtrPtr == NULL) { return OUTOFMEMORY; }
    (*nodePtrPtr)—>data = value;
    (*nodePtrPtr)—>left = NULL;
    (*nodePtrPtr)—>right = NULL;
    return INSERTED;
}
```

Ad es., per l'ARB:




inserire 44 produce l'ARB:



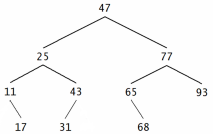
# Inserimento di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

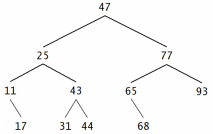
Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem.

```
/** @brief Assume: tree e' un ARB. Esegue: se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)—>data) { nodePtrPtr = &((*nodePtrPtr)—>left); }
        else if (value > (*nodePtrPtr)—>data) { nodePtrPtr = &((*nodePtrPtr)—>right); }
        else { return ALREADYPRESENT; }
    }
    *nodePtrPtr = malloc(sizeof(IntTreeNode));
    if (*nodePtrPtr == NULL) { return OUTOFMEMORY; }
    (*nodePtrPtr)—>data = value;
    (*nodePtrPtr)—>left = NULL;
    (*nodePtrPtr)—>right = NULL;
    return INSERTED;
}
```

Ad es., per l'ARB:




inserire 44 produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero.

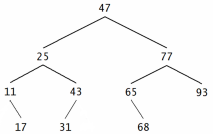
# Inserimento di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

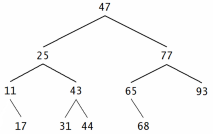
Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem.

```
/** @brief Assume: tree e' un ARB. Esegue: se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else { return ALREADYPRESENT; }
    }
    *nodePtrPtr = malloc(sizeof(IntTreeNode));
    if (*nodePtrPtr == NULL) { return OUTOFMEMORY; }
    (*nodePtrPtr)->data = value;
    (*nodePtrPtr)->left = NULL;
    (*nodePtrPtr)->right = NULL;
    return INSERTED;
}
```

Ad es., per l'ARB:




inserire 44 produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (`*tree` è vuoto o value è presente nella radice);
- $\Theta(h)$, nel caso pessimo (`value` è in una foglia di profondità massima, o non è presente e deve essere inserito come una foglia di profondità massima).

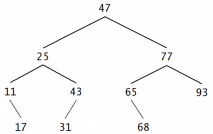
# Inserimento di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Inserimento di un elemento – `Outcome insertBSTiter(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

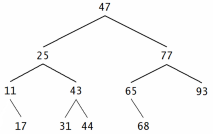
Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem.

```
/** @brief Assume: tree e' un ARB. Esegue: se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else { return ALREADYPRESENT; }
    }
    *nodePtrPtr = malloc(sizeof(IntTreeNode));
    if (*nodePtrPtr == NULL) { return OUTOFMEMORY; }
    (*nodePtrPtr)->data = value;
    (*nodePtrPtr)->left = NULL;
    (*nodePtrPtr)->right = NULL;
    return INSERTED;
}
```

Ad es., per l'ARB:




inserire 44 produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (`*tree` è vuoto o value è presente nella radice);
- $\Theta(h)$, nel caso pessimo (`value` è in una foglia di profondità massima, o non è presente e deve essere inserito come una foglia di profondità massima).

# ESTRAZIONE DEL MINIMO – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: toglie e restituisce il minimo elemento di un ARB non vuoto *treePtr
                (termina il programma con exit(-1) se *treePtr == NULL)
*/
T extractMinBSTiter(Tree *treePtr) {
    if (treePtr == NULL) { exit(-1); }

    Tree *nodePtrPtr = /* Valore che garantisce una "partenza corretta" */;

   // I.C.: Il minimo elemento di *treePtr e' il minimo elemento di *nodePtrPtr
   //**** (e se (*nodePtrPtr)->left == NULL, allora (*nodePtrPtr)->data e' il minimo elemento di *treePtr).
    while ( /* Condizione */ ) { /* Istruzioni che producono un avvicinamento alla soluzione
                                        (ovvero *nodePtrPtr punta al nodo che contiene il minimo) */ }

    /* Istruzioni che, assumendo (I.C. && !Condizione), eliminano il nodo che contiene il minimo e restituiscono il minimo */


}
```

# Estrazione del minimo – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: toglie e restituisce il minimo elemento di un ARB non vuoto *treePtr
                   (termina il programma con exit(-1) se *treePtr == NULL)
*/
T extractMinBSTiter(Tree *treePtr) {
    if (treePtr == NULL) { exit(-1); }

    Tree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: Il minimo elemento di *treePtr e' il minimo elemento di *nodePtrPtr
   //**** (e se (*nodePtrPtr)->left == NULL, allora (*nodePtrPtr)->data e' il minimo elemento di *treePtr).
    while ( /* Condizione */ ) { nodePtrPtr = &((*nodePtrPtr)->left); }


    /* Istruzioni che, assumendo (I.C. && !Condizione), eliminano il nodo che contiene il minimo e restituiscono il minimo */


}
```

# Estrazione del minimo – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: toglie e restituisce il minimo elemento di un ARB non vuoto *treePtr
                   (termina il programma con exit(-1) se *treePtr == NULL)
*/
T extractMinBSTiter(Tree *treePtr) {
    if (treePtr == NULL) { exit(-1); }

    Tree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: Il minimo elemento di *treePtr e' il minimo elemento di *nodePtrPtr
   //**** (e se (*nodePtrPtr)->left == NULL, allora (*nodePtrPtr)->data e' il minimo elemento di *treePtr).
    while ((*nodePtrPtr)->left != NULL) { nodePtrPtr = &((*nodePtrPtr)->left); }


    /* Istruzioni che, assumendo (I.C. && !Condizione), eliminano il nodo che contiene il minimo e restituiscono il minimo */


}
```

# Estrazione del minimo – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: toglie e restituisce il minimo elemento di un ARB non vuoto *treePtr
                   (termina il programma con exit(-1) se *treePtr == NULL)
*/
T extractMinBSTiter(Tree *treePtr) {
    if (treePtr == NULL) { exit(-1); }

    Tree *nodePtrPtr = treePtr;

   // I.C.: Il minimo elemento di *treePtr e' il minimo elemento di *nodePtrPtr
   //**** (e se (*nodePtrPtr)->left == NULL, allora (*nodePtrPtr)->data e' il minimo elemento di *treePtr).
    while ((*nodePtrPtr)->left != NULL) { nodePtrPtr = &((*nodePtrPtr)->left); }


    /* Istruzioni che, assumendo (I.C. && !Condizione), eliminano il nodo che contiene il minimo e restituiscono il minimo */


}
```

# Estrazione del minimo – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).

Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: toglie e restituisce il minimo elemento di un ARB non vuoto *treePtr
                (termina il programma con exit(-1) se *treePtr == NULL)
*/
T extractMinBSTiter(Tree *treePtr) {
    if (treePtr == NULL) { exit(-1); }

    Tree *nodePtrPtr = treePtr;

   // I.C.: Il minimo elemento di *treePtr e' il minimo elemento di *nodePtrPtr
   //**** (e se (*nodePtrPtr)->left == NULL, allora (*nodePtrPtr)->data e' il minimo elemento di *treePtr).
    while ((*nodePtrPtr)->left != NULL) { nodePtrPtr = &((*nodePtrPtr)->left); }

    Tree nodeToBeRemovedPtr = *nodePtrPtr;
    T minimum = nodeToBeRemovedPtr->data;
    *nodePtrPtr = nodeToBeRemovedPtr->right;
    free(nodeToBeRemovedPtr);
    return minimum;
}
```

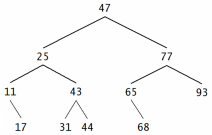
# Estrazione del minimo – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).

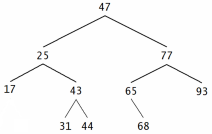
Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: toglie e restituisce il minimo elemento di un ARB non vuoto *treePtr
                   (termina il programma con exit(-1) se *treePtr == NULL)
*/
T extractMinBSTiter(Tree *treePtr) {
    if (treePtr == NULL) { exit(-1); }

    Tree *nodePtrPtr = treePtr;

   // I.C.: Il minimo elemento di *treePtr e' il minimo elemento di *nodePtrPtr
   //**** (e se (*nodePtrPtr)->left == NULL, allora (*nodePtrPtr)->data e' il minimo elemento di *treePtr).
    while ((*nodePtrPtr)->left != NULL) { nodePtrPtr = &((*nodePtrPtr)->left); }

    Tree nodeToBeRemovedPtr = *nodePtrPtr;
    T minimum = nodeToBeRemovedPtr->data;
    *nodePtrPtr = nodeToBeRemovedPtr->right;
    free(nodeToBeRemovedPtr);
    return minimum;
}
```

Ad es., per l'ARB:




ritorna 11 e produce l'ARB:



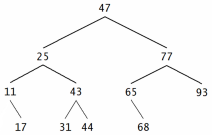
# Estrazione del minimo – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).

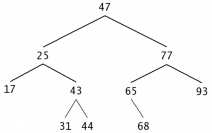
Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: toglie e restituisce il minimo elemento di un ARB non vuoto *treePtr
                   (termina il programma con exit(-1) se *treePtr == NULL)
*/
T extractMinBSTiter(Tree *treePtr) {
    if (treePtr == NULL) { exit(-1); }

    Tree *nodePtrPtr = treePtr;

   // I.C.: Il minimo elemento di *treePtr e' il minimo elemento di *nodePtrPtr
   //**** (e se (*nodePtrPtr)->left == NULL, allora (*nodePtrPtr)->data e' il minimo elemento di *treePtr).
    while ((*nodePtrPtr)->left != NULL) { nodePtrPtr = &((*nodePtrPtr)->left); }

    Tree nodeToBeRemovedPtr = *nodePtrPtr;
    T minimum = nodeToBeRemovedPtr->data;
    *nodePtrPtr = nodeToBeRemovedPtr->right;
    free(nodeToBeRemovedPtr);
    return minimum;
}
```

Ad es., per l'ARB:




ritorna 11 e produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero.

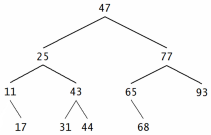
# Estrazione del minimo – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).

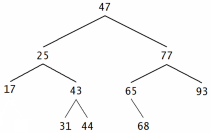
Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: toglie e restituisce il minimo elemento di un ARB non vuoto *treePtr
                   (termina il programma con exit(-1) se *treePtr == NULL)
*/
T extractMinBSTiter(Tree *treePtr) {
    if (treePtr == NULL) { exit(-1); }

    Tree *nodePtrPtr = treePtr;

   // I.C.: Il minimo elemento di *treePtr e' il minimo elemento di *nodePtrPtr
   //**** (e se (*nodePtrPtr)->left == NULL, allora (*nodePtrPtr)->data e' il minimo elemento di *treePtr).
    while ((*nodePtrPtr)->left != NULL) { nodePtrPtr = &((*nodePtrPtr)->left); }

    Tree nodeToBeRemovedPtr = *nodePtrPtr;
    T minimum = nodeToBeRemovedPtr->data;
    *nodePtrPtr = nodeToBeRemovedPtr->right;
    free(nodeToBeRemovedPtr);
    return minimum;
}
```

Ad es., per l'ARB:




ritorna 11 e produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (il minimo è nella radice);
- $\Theta(h)$, nel caso pessimo (il minimo è in una foglia di profondità massima).

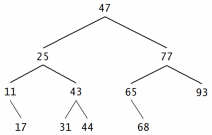
# Estrazione del minimo – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Estrazione del minimo – `T extractMinBSTiter(Tree *treePtr);` – elimina e restituisce il minimo elemento di un ARB non vuoto *treePtr (termina il programma se *treePtr == NULL).

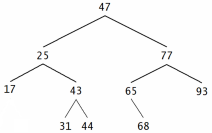
Basta osservare che in un ARB il nodo che contiene l'elemento minimo è il primo discendente sinistro della radice che non ha il figlio sinistro.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: toglie e restituisce il minimo elemento di un ARB non vuoto *treePtr
                   (termina il programma con exit(-1) se *treePtr == NULL)
*/
T extractMinBSTiter(Tree *treePtr) {
    if (treePtr == NULL) { exit(-1); }

    Tree *nodePtrPtr = treePtr;

   // I.C.: Il minimo elemento di *treePtr e' il minimo elemento di *nodePtrPtr
   //**** (e se (*nodePtrPtr)->left == NULL, allora (*nodePtrPtr)->data e' il minimo elemento di *treePtr).
    while ((*nodePtrPtr)->left != NULL) { nodePtrPtr = &((*nodePtrPtr)->left); }

    Tree nodeToBeRemovedPtr = *nodePtrPtr;
    T minimum = nodeToBeRemovedPtr->data;
    *nodePtrPtr = nodeToBeRemovedPtr->right;
    free(nodeToBeRemovedPtr);
    return minimum;
}
```

Ad es., per l'ARB:




ritorna 11 e produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (il minimo è nella radice);
- $\Theta(h)$, nel caso pessimo (il minimo è in una foglia di profondità massima).

Spazio: $\Theta(1)$, in tutti casi (ottimo, pessimo).

# RIMOZIONE DI UN ELEMENTO – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while /* Condizione */ ) {


        /* Istruzioni che producono un avvicinamento alla soluzione */




    }
    return /* Valore dedotto da (I.C. && !Condizione) */;
}
```

# Rimozione di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

Se il valore `value` da rimuovere è in una foglia o in un nodo che ha un solo figlio allora basta togliere il nodo dall'albero, altrimenti si può estrarre il minimo del sottoabero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

   // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while /* Condizione */ ) {


        /* Istruzioni che producono un avvicinamento alla soluzione */




    }
    return /* Valore dedotto da (I.C. && !Condizione) */;
}
```

# Rimozione di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

Se il valore `value` da rimuovere è in una foglia o in un nodo che ha un solo figlio allora basta togliere il nodo dall'albero, altrimenti si può estrarre il minimo del sottoabero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

  // I.C.: value e' presente in *treePtr se-e-solo-se e' presente in *nodePtrPtr
    while ( /* Condizione */ ) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else if (((*nodePtrPtr)->left != NULL) && ((*nodePtrPtr)->right != NULL)) {
            (*nodePtrPtr)->data = extractMinBSTiter(&((*nodePtrPtr)->right)); // Con T == int
            return 1;
        }
        else {
            Tree nodeToBeRemovedPtr = *nodePtrPtr;
            if ((*nodePtrPtr)->left == NULL) { *nodePtrPtr = (*nodePtrPtr)->right; }
            else { *nodePtrPtr = (*nodePtrPtr)->left; }
            free(nodeToBeRemovedPtr);
            return 1;
        }
    }
    return /* Valore dedotto da (I.C. && !Condizione) */;
}
```

# Rimozione di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

Se il valore `value` da rimuovere è in una foglia o in un nodo che ha un solo figlio allora basta togliere il nodo dall'albero, altrimenti si può estrarre il minimo del sottoabero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

  // I.C.: value e' presente in *treePtr se-e-solo-se e' presente in *nodePtrPtr
    while(*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else if (((*nodePtrPtr)->left != NULL) && ((*nodePtrPtr)->right != NULL)) {
            (*nodePtrPtr)->data = extractMinBSTiter(&((*nodePtrPtr)->right)); // Con T == int
            return 1;
        }
        else {
            Tree nodeToBeRemovedPtr = *nodePtrPtr;
            if ((*nodePtrPtr)->left == NULL) { *nodePtrPtr = (*nodePtrPtr)->right; }
            else { *nodePtrPtr = (*nodePtrPtr)->left; }
            free(nodeToBeRemovedPtr);
            return 1;
        }
    }
    return /* Valore dedotto da (I.C. && !Condizione) */;
}
```

# Rimozione di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

Se il valore `value` da rimuovere è in una foglia o in un nodo che ha un solo figlio allora basta togliere il nodo dall'albero, altrimenti si può estrarre il minimo del sottoabero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = /* Valore che garantisce una ''partenza corretta'' */;

  // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while(*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else if (((*nodePtrPtr)->left != NULL) && ((*nodePtrPtr)->right != NULL)) {
            (*nodePtrPtr)->data = extractMinBSTiter(&((*nodePtrPtr)->right)); // Con T == int
            return 1;
        }
        else {
            Tree nodeToBeRemovedPtr = *nodePtrPtr;
            if ((*nodePtrPtr)->left == NULL) { *nodePtrPtr = (*nodePtrPtr)->right; }
            else { *nodePtrPtr = (*nodePtrPtr)->left; }
            free(nodeToBeRemovedPtr);
            return 1;
        }
    }
    return 0;
}
```

# Rimozione di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

Se il valore `value` da rimuovere è in una foglia o in un nodo che ha un solo figlio allora basta togliere il nodo dall'albero, altrimenti si può estrarre il minimo del sottoabero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`.

---

```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

  // I.C.: value e' presente in *treePtr se-e-solo-se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else if (((*nodePtrPtr)->left != NULL) && ((*nodePtrPtr)->right != NULL)) {
            (*nodePtrPtr)->data = extractMinBSTiter(&((*nodePtrPtr)->right)); // Con T == int
            return 1;
        }
        else {
            Tree nodeToBeRemovedPtr = *nodePtrPtr;
            if ((*nodePtrPtr)->left == NULL) { *nodePtrPtr = (*nodePtrPtr)->right; }
            else { *nodePtrPtr = (*nodePtrPtr)->left; }
            free(nodeToBeRemovedPtr);
            return 1;
        }
    }
    return 0;
}
```

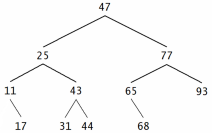
# Rimozione di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

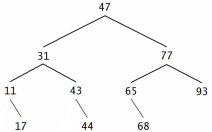
Se il valore `value` da rimuovere è in una foglia o in un nodo che ha un solo figlio allora basta togliere il nodo dall'albero, altrimenti si può estrarre il minimo del sottoabero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

  // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else if (((*nodePtrPtr)->left != NULL) && ((*nodePtrPtr)->right != NULL)) {
            (*nodePtrPtr)->data = extractMinBSTiter(&((*nodePtrPtr)->right)); // Con T == int
            return 1;
        }
        else {
            Tree nodeToBeRemovedPtr = *nodePtrPtr;
            if ((*nodePtrPtr)->left == NULL) { *nodePtrPtr = (*nodePtrPtr)->right; }
            else { *nodePtrPtr = (*nodePtrPtr)->left; }
            free(nodeToBeRemovedPtr);
            return 1;
        }
    }
    return 0;
}
```

Ad es., per l'ARB:




eliminare 25 produce l'ARB:



# Rimozione di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

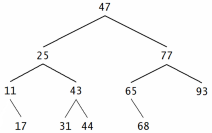
Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

Se il valore `value` da rimuovere è in una foglia o in un nodo che ha un solo figlio allora basta togliere il nodo dall'albero, altrimenti si può estrarre il minimo del sottoabero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`.

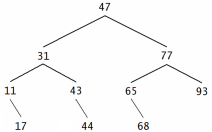
```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

  // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else if (((*nodePtrPtr)->left != NULL) && ((*nodePtrPtr)->right != NULL)) {
            (*nodePtrPtr)->data = extractMinBSTiter(&((*nodePtrPtr)->right)); // Con T == int
            return 1;
        }
        else {
            Tree nodeToBeRemovedPtr = *nodePtrPtr;
            if ((*nodePtrPtr)->left == NULL) { *nodePtrPtr = (*nodePtrPtr)->right; }
            else { *nodePtrPtr = (*nodePtrPtr)->left; }
            free(nodeToBeRemovedPtr);
            return 1;
        }
    }
    return 0;
}
```

Complessità computazionale rispetto all'altezza h dell'albero.

Ad es., per l'ARB:




eliminare 25 produce l'ARB:



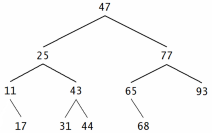
# Rimozione di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

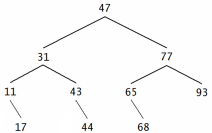
Se il valore `value` da rimuovere è in una foglia o in un nodo che ha un solo figlio allora basta togliere il nodo dall'albero, altrimenti si può estrarre il minimo del sottoabero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

  // I.C.: value e' presente in *treePtr se—e—solo—se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else if (((*nodePtrPtr)->left != NULL) && ((*nodePtrPtr)->right != NULL)) {
            (*nodePtrPtr)->data = extractMinBSTiter(&((*nodePtrPtr)->right)); // Con T == int
            return 1;
        }
        else {
            Tree nodeToBeRemovedPtr = *nodePtrPtr;
            if ((*nodePtrPtr)->left == NULL) { *nodePtrPtr = (*nodePtrPtr)->right; }
            else { *nodePtrPtr = (*nodePtrPtr)->left; }
            free(nodeToBeRemovedPtr);
            return 1;
        }
    }
    return 0;
}
```

Ad es., per l'ARB:




eliminare 25 produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (`*treePtr` è vuoto o value è presente nella radice);
- $\Theta(h)$, nel caso pessimo (`value` è in una foglia di profondità massima, o ha due figli e il minimo del suo sottoalbero destro è in una foglia di profondità massima, o non è presente e se lo fosse sarebbe in una foglia di profondità massima).

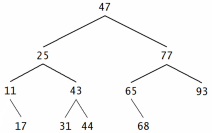
# Rimozione di un elemento – versione ITERATIVA (fa uso intensivo di puntatori a puntatori)

Rimozione di un elemento – `_Bool removeBSTiter(IntTree *treePtr, int value);` – rimuove value dall'ARB *treePtr e restituisce 1 se value è presente, altrimenti non modifica *treePtr e restituice 0.

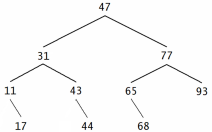
Se il valore `value` da rimuovere è in una foglia o in un nodo che ha un solo figlio allora basta togliere il nodo dall'albero, altrimenti si può estrarre il minimo del sottoabero destro (o il massimo del sottoalbero sinistro) e sostituirlo a `value`.

```
/** @brief Assume: *treePtr e' un ARB. Esegue: se value e' presente in *treePtr lo rimuove resituisce 1,
altrimenti non modifica *treePtr e restituisce 0.
*/
_Bool removeBSTiter(IntTree *treePtr, int value) {
    IntTree *nodePtrPtr = treePtr;

  // I.C.: value e' presente in *treePtr se-e-solo-se e' presente in *nodePtrPtr
    while (*nodePtrPtr != NULL) {
        if (value < (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->left); }
        else if (value > (*nodePtrPtr)->data) { nodePtrPtr = &((*nodePtrPtr)->right); }
        else if (((*nodePtrPtr)->left != NULL) && ((*nodePtrPtr)->right != NULL)) {
            (*nodePtrPtr)->data = extractMinBSTiter(&((*nodePtrPtr)->right)); // Con T == int
            return 1;
        }
        else {
            Tree nodeToBeRemovedPtr = *nodePtrPtr;
            if ((*nodePtrPtr)->left == NULL) { *nodePtrPtr = (*nodePtrPtr)->right; }
            else { *nodePtrPtr = (*nodePtrPtr)->left; }
            free(nodeToBeRemovedPtr);
            return 1;
        }
    }
    return 0;
}
```

Ad es., per l'ARB:




eliminare 25 produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (`*treePtr` è vuoto o value è presente nella radice);
- $\Theta(h)$, nel caso pessimo (`value` è in una foglia di profondità massima, o ha due figli e il minimo del suo sottoalbero destro è in una foglia di profondità massima, o non è presente e se lo fosse sarebbe in una foglia di profondità massima).

Spazio: $\Theta(1)$, in tutti casi (ottimo, pessimo).

# RICERCA DI UN ELEMENTO – versione RICORSIVA

Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBST(IntTree tree, int value) {
    if (tree == NULL) { return 0; }
    else if (value < tree->data) { return searchBST(tree->left, value); }
    else if (value > tree->data) { return searchBST(tree->right, value); }
    else { return 1; }
}
```

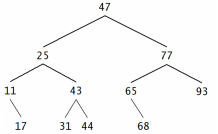
# Ricerca di un elemento – versione RICORSIVA

Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBST(IntTree tree, int value) {
    if (tree == NULL) { return 0; }
    else if (value < tree->data) { return searchBST(tree->left, value); }
    else if (value > tree->data) { return searchBST(tree->right, value); }
    else { return 1; }
}
```

Ad es., per l'ARB:




la ricerca di 42 restituisce 0, mentre la ricerca di 43 restituisce 1.

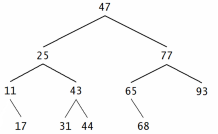
# Ricerca di un elemento – versione RICORSIVA

Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBST(IntTree tree, int value) {
    if (tree == NULL) { return 0; }
    else if (value < tree->data) { return searchBST(tree->left, value); }
    else if (value > tree->data) { return searchBST(tree->right, value); }
    else { return 1; }
}
```

Ad es., per l'ARB:




la ricerca di 42 restituisce 0, mentre la ricerca di 43 restituisce 1.

Complessità computazionale rispetto all'altezza h dell'albero.

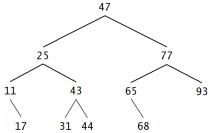
# Ricerca di un elemento – versione RICORSIVA

Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBST(IntTree tree, int value) {
    if (tree == NULL) { return 0; }
    else if (value < tree->data) { return searchBST(tree->left, value); }
    else if (value > tree->data) { return searchBST(tree->right, value); }
    else { return 1; }
}
```

Ad es., per l'ARB:




la ricerca di 42 restituisce 0, mentre la ricerca di 43 restituisce 1.

Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (value è nella radice);
- $O(h)$, nel caso pessimo (value non è presente o è nella foglia di profondità massima).

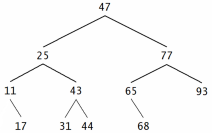
# Ricerca di un elemento – versione RICORSIVA

Ricerca di un elemento – `_Bool searchBST(IntTree tree, int value);` – restituisce 1 se value è presente, altrimenti restituisce 0.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. Restituisce: 1 se value e' presente in tree,
altrimenti restituisce 0
*/
_Bool searchBST(IntTree tree, int value) {
    if (tree == NULL) { return 0; }
    else if (value < tree->data) { return searchBST(tree->left, value); }
    else if (value > tree->data) { return searchBST(tree->right, value); }
    else { return 1; }
}
```

Ad es., per l'ARB:




la ricerca di 42 restituisce 0, mentre la ricerca di 43 restituisce 1.

Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (value è nella radice);
- $O(h)$, nel caso pessimo (value non è presente o è nella foglia di profondità massima).

Spazio: $\Theta(1)$, in tutti casi (ottimo, pessimo) se c'è la sibling call optizimation, altrimenti come per la complessità in tempo.

# INSERIMENTO DI UN ELEMENTO – versione RICORSIVA

Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBST(IntTree *treePtr, int value) {
    if (*treePtr == NULL) {
        *treePtr = malloc(sizeof(IntTreeNode));
        if (*treePtr == NULL) { return OUTOFMEMORY; }
        (*nodePtrPtr)->data = value;
        (*treePtr)->left = NULL;
        (*treePtr)->right = NULL;
        return INSERTED;
    }
  else if (value < (*treePtr)->data) { return insertBST(&(*treePtr)->left), value); }
  else if (value > (*treePtr)->data) { return insertBST(&(*treePtr)->right), value); }
  else { return ALREADYPRESENT; }
}
```

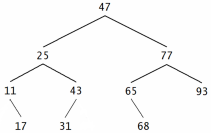
# Inserimento di un elemento – versione RICORSIVA

Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

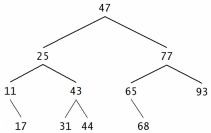
Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBST(IntTree *treePtr, int value) {
    if (*treePtr == NULL) {
        *treePtr = malloc(sizeof(IntTreeNode));
        if (*treePtr == NULL) { return OUTOFMEMORY; }
        (*nodePtrPtr)->data = value;
        (*treePtr)->left = NULL;
        (*treePtr)->right = NULL;
        return INSERTED;
    }
  else if (value < (*treePtr)->data) { return insertBST(&(*treePtr)->left), value); }
  else if (value > (*treePtr)->data) { return insertBST(&(*treePtr)->right), value); }
  else { return ALREADYPRESENT; }
}
```

Ad es., per l'ARB:




inserire 44 produce l'ARB:



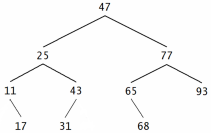
# Inserimento di un elemento – versione RICORSIVA

Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

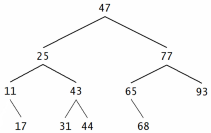
Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBST(IntTree *treePtr, int value) {
    if (*treePtr == NULL) {
        *treePtr = malloc(sizeof(IntTreeNode));
        if (*treePtr == NULL) { return OUTOFMEMORY; }
        (*nodePtrPtr)->data = value;
        (*treePtr)->left = NULL;
        (*treePtr)->right = NULL;
        return INSERTED;
    }
  else if (value < (*treePtr)->data) { return insertBST(&(*treePtr)->left), value); }
  else if (value > (*treePtr)->data) { return insertBST(&(*treePtr)->right), value); }
  else { return ALREADYPRESENT; }
}
```

Ad es., per l'ARB:




inserire 44 produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero.

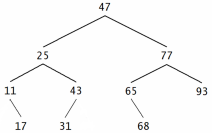
# Inserimento di un elemento – versione RICORSIVA

Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

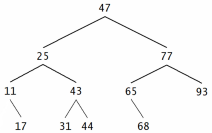
Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBST(IntTree *treePtr, int value) {
    if (*treePtr == NULL) {
        *treePtr = malloc(sizeof(IntTreeNode));
        if (*treePtr == NULL) { return OUTOFMEMORY; }
        (*nodePtrPtr)->data = value;
        (*treePtr)->left = NULL;
        (*treePtr)->right = NULL;
        return INSERTED;
    }
  else if (value < (*treePtr)->data) { return insertBST(&(*treePtr)->left), value); }
  else if (value > (*treePtr)->data) { return insertBST(&(*treePtr)->right), value); }
  else { return ALREADYPRESENT; }
}
```

Ad es., per l'ARB:




inserire 44 produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (`*treePtr` è vuoto o `value` è nella radice);
- $\Theta(h)$, nel caso pessimo (`value` è in una foglia di profondità massima, o non è presente e deve essere inserito come una foglia di profondità massima).

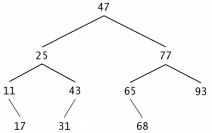
# Inserimento di un elemento – versione RICORSIVA

Inserimento di un elemento – `Outcome insertBST(IntTree *treePtr, int value);` – se value non è già presente, inserisce value nell'ARB *treePtr e restituisce INSERTED; altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY.

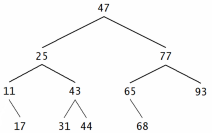
Basta ricordare che in un ARB, per ogni elemento (nodo) elem, il sottoalbero sinistro contiene elementi $\leq$ elem, e il sottoalbero destro contiene elementi $\geq$ elem .

```
/** @brief Assume: tree e' un ARB. se value non e' gia' presente, inserisce value nell'ARB *treePtr e restituisce INSERTED;
altrimenti non modifica *treePtr e restituice ALREADYPRESENT; se la malloc fallisce, restituisce OUTOFMEMORY
*/
Outcome insertBST(IntTree *treePtr, int value) {
    if (*treePtr == NULL) {
        *treePtr = malloc(sizeof(IntTreeNode));
        if (*treePtr == NULL) { return OUTOFMEMORY; }
        (*nodePtrPtr)->data = value;
        (*treePtr)->left = NULL;
        (*treePtr)->right = NULL;
        return INSERTED;
    }
  else if (value < (*treePtr)->data) { return insertBST(&(*treePtr)->left), value); }
  else if (value > (*treePtr)->data) { return insertBST(&(*treePtr)->right), value); }
  else { return ALREADYPRESENT; }
}
```

Ad es., per l'ARB:




inserire 44 produce l'ARB:




Complessità computazionale rispetto all'altezza h dell'albero. Tempo:

- $\Theta(1)$, nel caso ottimo (`*treePtr` è vuoto o `value` è nella radice);
- $\Theta(h)$, nel caso pessimo (`value` è in una foglia di profondità massima, o non è presente e deve essere inserito come una foglia di profondità massima).

Spazio: $\Theta(1)$, in tutti casi (ottimo, pessimo) se c'è la sibling call optizimation, altrimenti come per la complessità in tempo.

# Esercizi (in preparazione dell'esame)

# ESERCIZI (in preparazione dell'esame)

1. Scrivete P-IN e P-OUT per tutte le funzioni presentate in questo blocco di slides e per quelle che scriverete per i risolvere gli esercizi che seguono.

---

2. Scrivete una versione ricorsiva delle funzioni `findMinBSTiter`, `extractMinBSTiter`, e `removeBSTiter` (chiamatele `findMinBST`, `extractMinBST`, e `removeBST`).
3. Scrivete una versione delle funzioni `removeBSTiter` e `removeBST` (chiamatele `altRemoveBSTiter` e `altRemoveBST`) che, per eliminare un elemento con due figli, lo sostituiscono con il massimo del sottoalbero sinistro.

---

4. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `countNodesIter` e una funzione ricorsiva `countNodes` che contino il numero di nodi di un albero binario.
5. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `countLeavesIter` e una funzione ricorsiva `countLeaves` che contino il numero di foglie di un albero binario.
6. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `countInnerNodesIter` e una funzione ricorsiva `countInnerNodes` che contino il numero di nodo interni di un albero binario.
7. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `heightIter` e una funzione ricorsiva `height` che calcolino l'altezza di un albero binario.
8. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `isCompleteIter` e una funzione ricorsiva `isComplete` che controllino se un albero binario è completo.
9. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `isAlmostCompleteIter` e una funzione ricorsiva `isAlmostComplete` che controllino se un albero binario è quasi completo.
10. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `isPseudoAlmostCompleteIter` e una funzione ricorsiva `isPseudoAlmostComplete` che controllino se un albero binario è pseudo quasi completo.
11. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `isDegenerateIter` e una funzione ricorsiva `isDegenerate` che controllino se un albero binario è degenere.
12. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `isCompletelyUnbalancedToTheLeftIter` e una funzione ricorsiva `isCompletelyUnbalancedToTheLeft` che controllino se un albero binario è completamente sbilanciato a sinistra.
13. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `isCompletelyUnbalancedLeftIter` e una funzione ricorsiva `isCompletelyUnbalanced` che controllino se un albero binario è completamente sbilanciato.

---

14. Scrivete, sviluppandole assieme alla dimostrazione di correttezza, una funzione iterativa `isBSTiter` e una funzione ricorsiva `isBST` che controllino se un albero binario è un ARB.

---

15. Scrivete, sviluppandola assieme alla dimostrazione di correttezza, una implementazione `VoidStarBSTSortedSet` dell'ADT `VoidStarSortedSet` e testatela con elementi di tipo string (`char*`) e `ContactPtr`.

# Take-home message(s)

# Cose da studiare e cose da leggere

- Cose da studiare (entro la prossima Lezione):
  1. Questi lucidi

Se non vi fosse chiaro qualcosa, chiedete asap: se avrò una risposta, ve la comunicherò – altrimenti vi dirò che non ce l'ho e continuerò pensarci (di tanto in tanto) almeno fino a quando non ne troverò una (anche se dovessi farlo per tutto il resto della vita); nel caso, voi fate altrettanto 😉.

# Take-home message(s)

1. In generale, la versione ricorsiva degli algoritmi che operano su strutture dati ricorsive è più facila da scrivere (assieme alla dimostrazione di correttezza, usando il **principio di induzione strutturale**) e da leggere.
2. La versione iterativa può avere complessità computazionale in spazio minore, ed essere più performante – cercate di usare la ricorsione di coda e controllate quali ottimizzazioni supporta il vostro compilatore C.

5

# Take-home message(s)

1. In generale, la versione ricorsiva degli algoritmi che operano su strutture dati ricorsive è più facila da scrivere (assieme alla dimostrazione di correttezza, usando il **principio di induzione strutturale**) e da leggere.
2. La versione iterativa può avere complessità computazionale in spazio minore, ed essere più performante – cercate di usare la ricorsione di coda e controllate quali ottimizzazioni supporta il vostro compilatore C.
3. Il bilanciamento (ovvero l'altezza) di un ARB creato per inserimenti successivi con la funzione `insertBSTiter` (o `insertBST`) può variare molto a seconda dell'ordine in cui sono inseriti gli elementi.

5

# Take-home message(s)

1. In generale, la versione ricorsiva degli algoritmi che operano su strutture dati ricorsive è più facila da scrivere (assieme alla dimostrazione di correttezza, usando il **principio di induzione strutturale**) e da leggere.
2. La versione iterativa può avere complessità computazionale in spazio minore, ed essere più performante – cercate di usare la ricorsione di coda e controllate quali ottimizzazioni supporta il vostro compilatore C.
3. Il bilanciamento (ovvero l'altezza) di un ARB creato per inserimenti successivi con la funzione `insertBSTiter` (o `insertBST`) può variare molto a seconda dell'ordine in cui sono inseriti gli elementi. Ad esempio, inserendo gli elementi in modo strettamente crescente si ottiene un albero completamente sbilanciato a destra.

5

# Take-home message(s)

1. In generale, la versione ricorsiva degli algoritmi che operano su strutture dati ricorsive è più facila da scrivere (assieme alla dimostrazione di correttezza, usando il **principio di induzione strutturale**) e da leggere.
2. La versione iterativa può avere complessità computazionale in spazio minore, ed essere più performante – cercate di usare la ricorsione di coda e controllate quali ottimizzazioni supporta il vostro compilatore C.
3. Il bilanciamento (ovvero l'altezza) di un ARB creato per inserimenti successivi con la funzione `insertBSTiter` (o `insertBST`) può variare molto a seconda dell'ordine in cui sono inseriti gli elementi. Ad esempio, inserendo gli elementi in modo strettamente crescente si ottiene un albero completamente sbilanciato a destra.

   Esistono delle versioni bilanciate degli ARB, ovvero ARB in cui le operazioni di inserimento, cancellazione e ricerca hanno sempre complessità $O(\log_2 n)$ (dove $n$ è il numero dei nodi dell'ARB) – in particolare, le operazioni di inserimento e cancellazione cambiano la struttura dell'ARB.

5

# Take-home message(s)

1. In generale, la versione ricorsiva degli algoritmi che operano su strutture dati ricorsive è più facila da scrivere (assieme alla dimostrazione di correttezza, usando il **principio di induzione strutturale**) e da leggere.
2. La versione iterativa può avere complessità computazionale in spazio minore, ed essere più performante – cercate di usare la ricorsione di coda e controllate quali ottimizzazioni supporta il vostro compilatore C.
3. Il bilanciamento (ovvero l'altezza) di un ARB creato per inserimenti successivi con la funzione `insertBSTiter` (o `insertBST`) può variare molto a seconda dell'ordine in cui sono inseriti gli elementi. Ad esempio, inserendo gli elementi in modo strettamente crescente si ottiene un albero completamente sbilanciato a destra.

   Esistono delle versioni bilanciate degli ARB, ovvero ARB in cui le operazioni di inserimento, cancellazione e ricerca hanno sempre complessità $O(\log_2 n)$ (dove $n$ è il numero dei nodi dell'ARB) – in particolare, le operazioni di inserimento e cancellazione cambiano la struttura dell'ARB.

   Le versioni più famose di tali ARB,[5] sono (in ordine di apparizione):

[5] Che probalmente vi saranno presentate nell'Insegnamento di Algoritmi e Strutture Dati.

# Take-home message(s)

1. In generale, la versione ricorsiva degli algoritmi che operano su strutture dati ricorsive è più facila da scrivere (assieme alla dimostrazione di correttezza, usando il **principio di induzione strutturale**) e da leggere.
2. La versione iterativa può avere complessità computazionale in spazio minore, ed essere più performante – cercate di usare la ricorsione di coda e controllate quali ottimizzazioni supporta il vostro compilatore C.
3. Il bilanciamento (ovvero l'altezza) di un ARB creato per inserimenti successivi con la funzione `insertBSTiter` (o `insertBST`) può variare molto a seconda dell'ordine in cui sono inseriti gli elementi. Ad esempio, inserendo gli elementi in modo strettamente crescente si ottiene un albero completamente sbilanciato a destra.

   Esistono delle versioni bilanciate degli ARB, ovvero ARB in cui le operazioni di inserimento, cancellazione e ricerca hanno sempre complessità $O(\log_2 n)$ (dove $n$ è il numero dei nodi dell'ARB) – in particolare, le operazioni di inserimento e cancellazione cambiano la struttura dell'ARB.

   Le versioni più famose di tali ARB,[5] sono (in ordine di apparizione):

   - Gli **alberi AVL** – **AVL trees** (inventati nel 1962 da Georgy **A**delson-**V**elsky e Evgenii **L**andis).
   - Gli **alberi rosso-neri** – **red-black trees** (inventati nel 1978 da Leonidas J. Guibas e Robert Sedgewick).

[5] Che probalmente vi saranno presentate nell'Insegnamento di Algoritmi e Strutture Dati.